Projet interdisciplinaire

Groupe 5 L3 MIASHS, parcours MIAGE, 2025-2026 :
BABACHANAKH Kateryna
SHKRED Diana
CIFTCI Mehmet Can
SAIDANI Alyssa

TABLE DES MATIÈRES

INTRODUCTION..... 3
I. PARTIE GESTION..... 4
1. La Note de Cadrage..... 4
1.1. Titre..... 4
1.2. Contexte du projet..... 4
1.3. Objectif général..... 4
1.4. Données d'entrée (input)..... 5
1.5. Périmètre de notre mission..... 5
1.6. Produit du projet..... 6
1.7. Objectifs de performances, de coûts, de délais..... 6
1.8. Acteurs du projet..... 6
1.9. Contraintes..... 7
1.10. Risques..... 7
2. PBS (Product Breakdown Structure)..... 10
3. WBS (Work Breakdown Structure)..... 12
4. Diagramme de Gantt..... 13
5. Tableau Trello..... 14
II. PARTIE COMPILATION..... 16
1. Suite de tests..... 16
2. Grammaire..... 18
2.1. La grammaire CUP de notre langage..... 18
2.2. La grammaire enrichie du langage..... 21
3. Évaluation..... 22
Exemple 1 : Programme minimal..... 22
Exemple 2 : Variables globales..... 23
Exemple 3 : Expression..... 24
Exemple 4 : Ecriture..... 26
Exemple 5 : Ecriture..... 28
Exemple 6 : Variables locales et paramètres..... 30
Exemple 7 : Conditionnelle..... 33
Exemple 8 : Itération..... 35
Exemple 9 : Récursivité..... 37
Bilan critique..... 40
4. Distribution binaire du compilateur..... 41
CONCLUSION..... 42
ANNEXES..... 43

INTRODUCTION

Dans le cadre de notre formation en L3 MIASHS, parcours MIAGE, nous avons réalisé le projet COMPILEXXL, qui consiste à concevoir un compilateur pour un langage impératif simple vers l'assembleur BETA. Ce projet interdisciplinaire s'inscrit dans le cadre des cours de compilation et de gestion de projet. Il nous a permis de mettre en œuvre des compétences techniques (analyse lexicale et syntaxique, génération de code) ainsi que des compétences organisationnelles (planification, gestion des tâches, travail en équipe).

L'objectif principal était de comprendre et implémenter les différentes étapes de la compilation, notamment l'analyse lexicale avec JFlex, l'analyse syntaxique avec CUP, la construction d'un arbre abstrait, la gestion d'une table des symboles, ainsi que la génération de code assembleur exécutable avec BSim.

Le projet a été réalisé en équipe utilisant une approche progressive et structurée, en utilisant des outils de gestion tels que PBS, WBS, le diagramme de Gantt et Trello pour organiser le travail et suivre les progrès. Par ailleurs, l'ensemble du code source ainsi que le suivi du projet ont été centralisés sur GitHub, accessible via le lien suivant : https://github.com/babachanakh-kateryna/project-compilexxl-l3.git. Cela nous a permis d'assurer une collaboration efficace, une traçabilité du travail et une gestion des versions.

Ce rapport se compose de deux parties principales. La première partie est consacrée à la gestion du projet, incluant la note de cadrage, la planification (PBS, WBS, Gantt) et le suivi des tâches via Trello. La seconde partie présente la réalisation technique du compilateur, notamment la définition de la grammaire, l'analyse syntaxique, la construction des structures internes et l'évaluation à travers un ensemble de programmes de test. Enfin, un bilan critique et les annexes.

I. PARTIE GESTION

1. La Note de Cadrage

1.1. Titre

COMPILEXXL - Compilateur d'un langage impératif simple vers l'assembleur BETA

1.2. Contexte du projet

1.2.1. Origine

Ce projet est en lien avec notre cours universitaire de Compilation L3 MIASHS. Il a été demandé par Azim Roussanaly en lien avec le cours de Gestion de projet par Rachel ALTMEYER.

1.2.2. Enjeux

Maîtriser la chaîne de compilation complète, en particulier l'analyse syntaxique et la génération de code assembleur BETA. Appliquer une démarche agile avec 9 points de progression progressifs (du programme minimal aux appels récursifs). Pour l'étape 2, l'enjeu principal est de concevoir un analyseur syntaxique à l'aide de CUP, capable de construire automatiquement un arbre abstrait et une table des symboles, puis de réutiliser ces structures pour générer un code assembleur valide.
Respecter les échéances du projet, notamment la date limite de l'étape 1 (17 février 2026) ainsi que la livraison finale prévue pour le 6 avril 2026.

1.2.3. Besoins client

Le client a besoin d'un compilateur capable de traduire un programme source écrit dans un langage impératif simple vers du code assembleur BETA exécutable. Pour l'étape 1 : implémenter le package GENERATION qui produit du code assembleur BETA à partir de l'arbre abstrait syntaxique et de la table des symboles (TDS). Pour l'étape 2, le compilateur doit être capable d'analyser syntaxiquement un programme source, de construire automatiquement l'arbre abstrait, de générer la table des symboles, et de produire du code assembleur correct en lien avec l'étape 1.

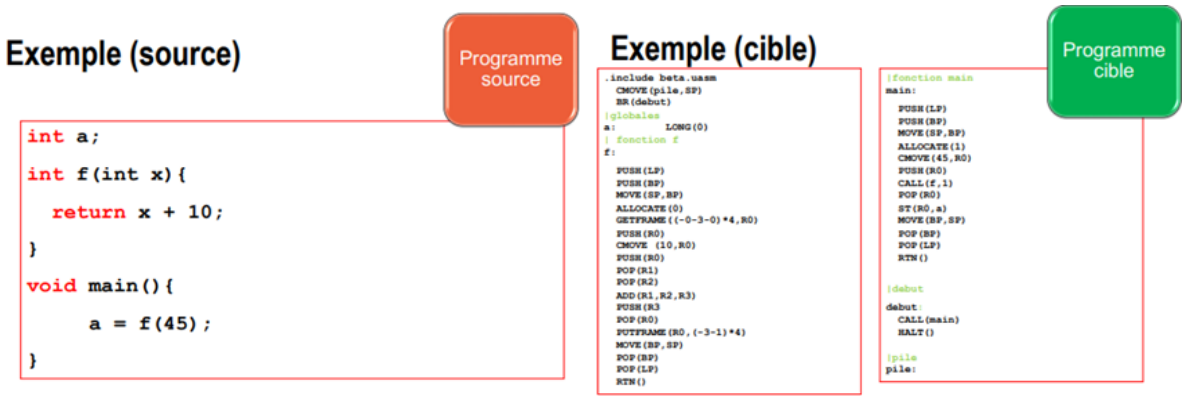
1.3. Objectif général

Développer un compilateur complet transformant un programme source en code BETA exécutable.

- ➢ Pour l'étape 1 : concevoir et implémenter la génération de code assembleur pour 9 exemples progressifs (variables globales, expressions, écriture, variables locales/paramètres, conditionnelles, itérations, récursivité).
- ➢ Pour l'étape 2 : concevoir et implémenter un analyseur syntaxique basé sur CUP, permettant de construire automatiquement l'arbre abstrait et la table des symboles à partir des programmes du jeu de tests.

Le compilateur doit être capable de traiter les 9 exemples définis (programme minimal, variables globales, expressions, écriture, variables locales et paramètres, conditionnelles, itérations, récursivité) et de générer un code valide exécuté sous BSim.
Selon le site d' IBM : « Les compilateurs sont utilisés pour transformer le code source de haut niveau en code cible de bas niveau ».
Selon le client Azim ROUSSANALY: « Pour un être humain c'est difficile d'exécuter en binaire contrairement au programme source. Celui-ci permettra de traduire dans un langage compris une résolution de problème.

Le passage du programme source à une exécution de code binaire se passe grâce à un compilateur. »



Plus simplement, les images ci-dessus illustrent bien : un code/programme source représente le langage de programmation « connu » par les informaticiens. Ce sont leurs outils de travail et leur langage habituel. Le code/programme cible permet au programme source de fonctionner, c'est un langage plus compliqué à comprendre qui nécessite beaucoup de travail pour l'informaticien car il n'a pas l'habitude de coder avec celui-ci.

1.4. Données d'entrée (input)

1. De nos connaissances en « grammaire » vu en cours de langage formel avec Romain Péchoux (qu'on verra avec le logiciel CUP qui sera abordée pendant le cours de COMPILATION) ;
2. Expression régulière vu par Romain Péchoux (qu'on verra avec le logiciel JFlex) ;
3. Expertise JAVA que nous avons travaillé tout au long de notre formation ;
4. Maîtriser le langage cible Assembleur BETA. Nous aurons donc besoin des connaissances vu en cours d'architecture des ordinateurs avec Romain Péchoux pour le côté « assembleur » ;
5. Packages TDS (Table des Symboles) et ARBRE vus en TD pour la génération de code ;
6. Jeu de test avec 9 exemples concrets (programme minimal, variables globales, expressions, écriture, variables locales et paramètres, conditionnelles, itération, récursivité).
7. Bsim pour exécuter et tester le code BETA généré.

1.5. Périmètre de notre mission

1.5.1. Ce qu'on fera ce qu'on fera :

1. dépôt sur GitHub ;
2. définition de la syntaxe et des règles du langage impératif ;
3. écriture de la grammaire CUP du langage ;
4. enrichissement de la grammaire avec du code Java pour construire automatiquement la TDS et l'arbre abstrait ;
5. utilisation des outils JFlex et CUP pour l'analyse lexicale et syntaxique ;
6. déclaration des terminaux/non-terminaux et définition des priorités de syntaxe ;
7. construction automatique de la TDS et de l'Arbre Abstrait ;
8. génération du code assembleur BETA correct et exécutable ;
9. développement complet du compilateur en Java ;
10. implémentation des vérifications (types corrects, variables déclarées avant utilisation, opérateurs compatibles avec les types, etc.) ;
11. implémentation du package GENERATION : algorithmes pour générer assembleur BETA à partir de l'arbre abstrait et TDS, testés point par point (9 étapes agiles) ;
12. validation du code avec BSim.

1.5.2. Ce qu'on fera ce qu'on ne fera pas :

1. implémenter des types complexes (tableaux, objets, chaînes, etc.) ;
2. plus d'instruction que Affectation / Alternative / Répétitive / Appel d'un sous-programme / Ecriture ;
3. d'autres langages que celui impératif ;
4. l'optimisation du code assembleur (le programme généré ne sera pas optimisé pour être plus petit ou plus rapide) ;
5. d'autre programme cible que le Beta Assembleur ;
6. interface graphique pour le compilateur ;
7. la transformation de l'assembleur en programme objet (éditeur de liens).

1.6. Produit du projet

Le produit final du projet est un compilateur développé en Java pour un langage impératif simple et minimal orienté apprentissage, centré sur le type entier, les opérations arithmétiques et relationnelles (+, -, *, /, >, <, >=, <=, ==, !=), les instructions de contrôle (affectation, alternative, répétitive), et les sous-programmes (fonctions/procédures avec paramètres, variables locales et appels récursifs).

Les livrables pour le premier étape :

1. Le rapport : pour chaque exemple du jeu de test : arbre abstrait, table des symboles, code généré, exécution du code avec Bsim, évaluation (comparaison avec le code attendu).
2. Le code source Java (package GENERATION).

Les livrables pour l'étape 2 :

1. un analyseur syntaxique basé sur CUP ;
2. une grammaire définissant le langage ;
3. une grammaire enrichie permettant de construire automatiquement l'arbre abstrait et la TDS
4. la génération du code assembleur BETA à partir de ces structures ;
5. un jeu de tests composé de 9 programmes sources ;
6. un rapport détaillant les résultats (arbre abstrait, TDS, code généré, exécution sous BSim, évaluation) ;
7. l'ensemble du code source du projet.

1.7. Objectifs de performances, de coûts, de délais

Le compilateur doit compiler sans erreur les programmes conformes aux spécifications et rejeter proprement les programmes erronés avec des messages clairs. Le coût du projet sera calculé en fonction de l'effort et du temps de développement. Sa réalisation est prévue dans un délai global de deux mois, mais le projet sera divisé en plusieurs étapes progressives.

Pour l'étape 1 : code BETA généré correctement et exécutable avec Bsim pour les 9 exemples, avec date limite étape 1 : 17 février 2026 (23:55).

Pour l'étape 2 : le parseur doit analyser correctement tous les programmes du jeu de tests, l'arbre abstrait et la TDS doivent être construits automatiquement, le code généré doit être conforme à celui validé lors de l'étape 1, le programme doit être exécutable sans erreur dans BSim.

1.8. Acteurs du projet

Maître d'ouvrage : Azim ROUSSANALY

Maître d'oeuvre : Diana SHKRED, Mehmet Can CIFTCI, Kateryna BABACHANAKH

Comité de pilotage : Azim ROUSSANALY, Rachel ALTMEYER, Diana SHKRED, Mehmet Can CIFTCI, Kateryna BABACHANAKH

Chef de projet : Kateryna BABACHANAKH

1.9. Contraintes

1. utilisation obligatoire du langage Java ;
2. utilisation de JFlex et CUP ;
3. génération de code en assembleur BETA ;
4. respect des consignes pédagogiques et du format attendu ;
5. respect du calendrier universitaire et des règles d'évaluation (travail en équipe, pas de plagiat, pas d'IA) ;
6. rendre plusieurs versions du projet au client pour qu'il puisse voir l'avance ;
7. gestion des types de données seront seulement des entiers. (on ne fera pas d'autres types de données que les entiers) ;
8. l'implémentation d'opérations arithmétiques et de comparaison (+, -, *, /, >, =, <=, ==, ! =) ;
9. la mise en œuvre d'instructions de base : affectation, alternatives, répétitions et appels de sous-programmes (fonctions ou procédures) ;
10. le support de variables locales, de paramètres et d'appels récursifs ;
11. travail par équipe-projet, un seul rapport par équipe, approche agile avec livrables par point de progression.

1.10. Risques

Conformément à la méthodologie du projet, nous définissons un risque comme un événement dont l'apparition n'est pas certaine et dont la manifestation est susceptible d'affecter les objectifs du projet (qualité, délais, conformité).

1.10.1. Risques liés au Projet

Produit

Risque : non-conformité du produit aux attentes pédagogiques et techniques
Le compilateur développé pourrait ne pas respecter totalement les spécifications attendues (fonctionnalités obligatoires, génération correcte du code BETA, conformité aux exemples fournis).
Solutions / mesures :

- Respect strict des consignes du cours et du périmètre défini.
- Tests systématiques.
- Validation régulière du code généré avec Bsim.
- Communication avec le client pour s'assurer des attentes

Budget

Risque : sous-estimation de l'effort nécessaire
L'arriver des vacances qui peuvent nous stopper dans le projet, stopper la communication pour son avancer, et perdre de vue nos objectifs s'ils ne sont pas bien définis dans le PBS et WBS.
Même si le projet ne comporte pas de coût financier direct, le temps et l'énergie des membres de l'équipe peuvent être sous-estimés, ce qui peut impacter la qualité du rendu.
Solutions / mesures :

- Planification réaliste des tâches.

- Répartition équitable de la charge de travail.
- Priorisation des fonctionnalités obligatoires avant toute amélioration facultative.

Compétences

Risque : difficultés techniques liées aux compétences requises
Le projet mobilise des compétences avancées en Java, JFlex, CUP et assembleur BETA, qui peuvent ne pas être parfaitement maîtrisées par tous les membres.
Solutions / mesures :

- Appui sur les cours, TD et supports pédagogiques.
- Partage de connaissances au sein de l'équipe.
- Avancement progressif, du plus simple au plus complexe.

Clients

Risque : mauvaise compréhension des attentes du client (enseignant)
Une interprétation incorrecte des attentes pédagogiques peut conduire à un livrable non conforme.
Solutions / mesures :

- Références systématiques aux consignes officielles.
- Échanges réguliers avec le client et le comité de pilotage.
- Livraison de versions intermédiaires pour validation.

1.10.2. Risques liés à l'Organisation

Processus décisionnels et hiérarchie

Risque : lenteur ou confusion dans la prise de décision
Une absence de hiérarchie claire ou de processus décisionnel peut ralentir l'avancement du projet.
Solutions / mesures :

- Définition claire des rôles (chef de projet, développeurs).
- Centralisation des décisions techniques importantes.
- Arbitrage rapide en cas de blocage.

Communication

Risque : manque de communication au sein de l'équipe
Des problèmes de communication peuvent entraîner des erreurs, des doublons ou des retards.
Solutions / mesures :

- Réunions régulières de suivi.
- Utilisation d'outils communs (GitHub) pour centraliser le travail.
- Documentation des choix techniques et organisationnels.

1.10.3. Risques liés à l'Environnement

Normes et lois

Risque : non-respect des règles universitaires et pédagogiques
Le projet est soumis à des règles strictes (langage imposé, outils obligatoires, interdiction de l'IA, règles de plagiat).

Solutions / mesures :

- Respect rigoureux des consignes fournies.
- Travail original et traçable via GitHub.
- Vérification régulière de la conformité du projet.

Concurrence

Risque : comparaison défavorable avec les autres groupes
D'autres équipes peuvent produire un travail plus avancé ou plus propre, ce qui peut impacter l'évaluation.
Solutions / mesures :

- Respect des attendus plutôt que la surenchère fonctionnelle.
- Qualité du code, clarté du rapport et conformité aux exemples.
- Approche progressive et maîtrisée.

Marché

Risque : inadéquation avec le contexte académique
Le projet n'est pas destiné à un usage industriel mais à un cadre pédagogique précis.
Solutions / mesures :

- Se limiter strictement au contexte universitaire.
- Ne pas chercher à complexifier inutilement le compilateur.
- Prioriser l'aspect pédagogique et démonstratif.

2. PBS (Product Breakdown Structure)

Dans le cadre du projet COMPILEXXL, le PBS permet d'identifier clairement les composants du compilateur ainsi que les livrables à produire. Il sert de base à l'organisation du projet et à la planification des tâches (WBS, diagramme de Gantt, Trello). Le projet est structuré autour de quatre grandes catégories principales, qui représentent les composantes essentielles du projet :

1. Les livrables
2. La gestion de projet
3. La partie technique
4. Le code source Java

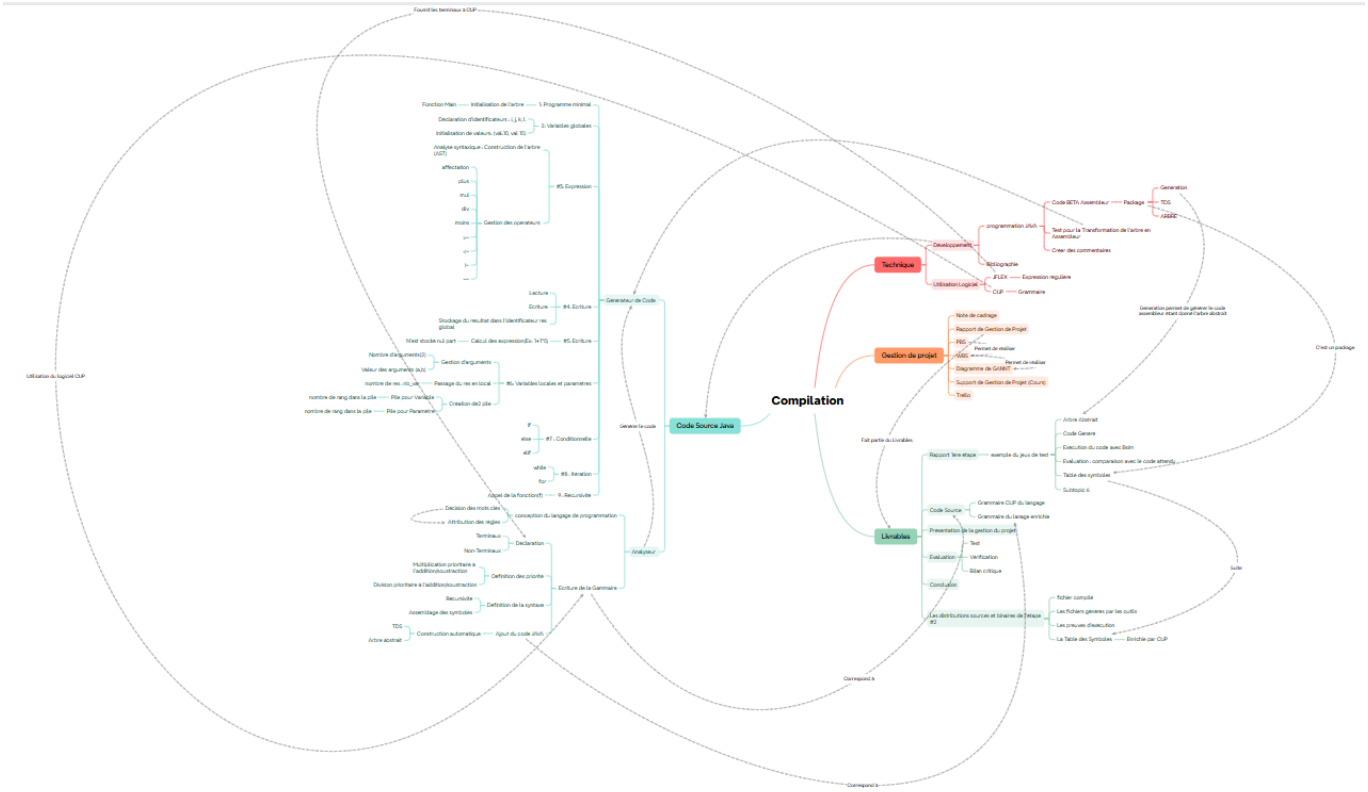


Figure 1 : Structure globale du projet COMPILEXXL (PBS)

Sur la figure présentée, on peut observer que le projet possède une organisation centralisée, articulée autour de quatre branches principales rayonnant du noyau « Compilation ». Ces quatre branches correspondent précisément aux catégories suivantes : les livrables, la gestion de projet, la partie technique et le code source Java.

Cependant, cette vue est difficile à appréhender en détail. Pour une meilleure compréhension, des versions plus détaillées de chaque section sont présentées en annexes.

1. Les livrables

Les livrables correspondent aux éléments remis à la fin du projet :

- ➢ le rapport de l'étape 1 ;
- ➢ le rapport final (étape 2) ;
- ➢ la distribution du projet (code source et version exécutable du compilateur).

Ces livrables permettent de valider le projet dans le cadre académique (voir détails en Annexe 1).

2. La gestion de projet

Cette partie regroupe les outils et documents utilisés pour organiser le travail :

- ➢ la note de cadrage, qui définit les objectifs, le périmètre et les contraintes ;
- ➢ le PBS, le WBS et le diagramme de Gantt pour la planification ;
- ➢ Trello pour le suivi des tâches et la coordination de l'équipe.

(voir détails en Annexe 2).

3. La partie technique

La partie technique concerne les outils et l'environnement du compilateur :

- ➢ JFlex pour l'analyse lexicale ;
- ➢ CUP pour l'analyse syntaxique et la construction de l'arbre abstrait et de la TDS ;
- ➢ BSim pour l'exécution du code assembleur BETA.

(voir détails en Annexe 2).

4. Le code source Java

Le code source constitue le cœur du projet et comprend :

- ➢ la construction de l'arbre abstrait ;
- ➢ la gestion de la table des symboles ;
- ➢ la génération du code assembleur BETA ;
- ➢ le fonctionnement global du compilateur (analyse → arbre abstrait/TDS → génération)

(voir détails en Annexe 3).

Ces éléments permettent de réaliser un compilateur fonctionnel capable de traiter les programmes du jeu de tests.

3. WBS (Work Breakdown Structure)

Le WBS (Work Breakdown Structure) reprend la structure hiérarchique du PBS en y ajoutant des informations nécessaires à la planification et au suivi du projet (voir Annexe 4). Dans le cadre du projet COMPILEXXL, le WBS permet d'organiser concrètement le travail en précisant, pour chaque tâche :

- ➢ les responsables ;
- ➢ les dates de début et de fin estimées ;
- ➢ les dates réelles ;
- ➢ le pourcentage d'avancement.

Le WBS a été construit directement à partir du PBS en conservant les mêmes grandes catégories :

1. livrables ;
2. gestion de projet ;
3. partie technique ;
4. code source Java.

Chaque catégorie a ensuite été décomposée en tâches et sous-tâches. Une répartition du travail a été réalisée entre les membres du groupe en fonction des compétences et de la charge de travail. Une fois les tâches réparties nous avons estimé pour chaque étape une date de début estimée et une durée estimée. Ces estimations ont été réalisées collectivement en tenant compte de la complexité des tâches et des dépendances entre certaines étapes du projet. Nous avons dû prendre plusieurs semaines/mois pour finir le WBS car nous n'avions pas eu toutes les étapes du projet directement.
La planification a pris en compte les dépendances entre les différentes étapes du projet. Par exemple, la génération de code dépend de la construction de l'arbre abstrait et de la table des symboles.

Au fur et à mesure de l'avancement du projet, les informations initiales du WBS ont été complétées par la date de début réelle et la durée réelle. Cela permet de comparer la planification initiale avec la progression réelle du projet et d'identifier les éventuels écarts entre les prévisions et la réalisation. Ce suivi permet également d'avoir une vision précise de l'état d'avancement du projet à tout moment et de voir le retard qui pourrait potentiellement s'accumuler (Il y a un lien fort avec le cours de recherche opérationnelle que nous avons eu ce semestre).

Le WBS sert également de base à la construction du diagramme de Gantt et à l'organisation du suivi sur Trello.

4. Diagramme de Gantt

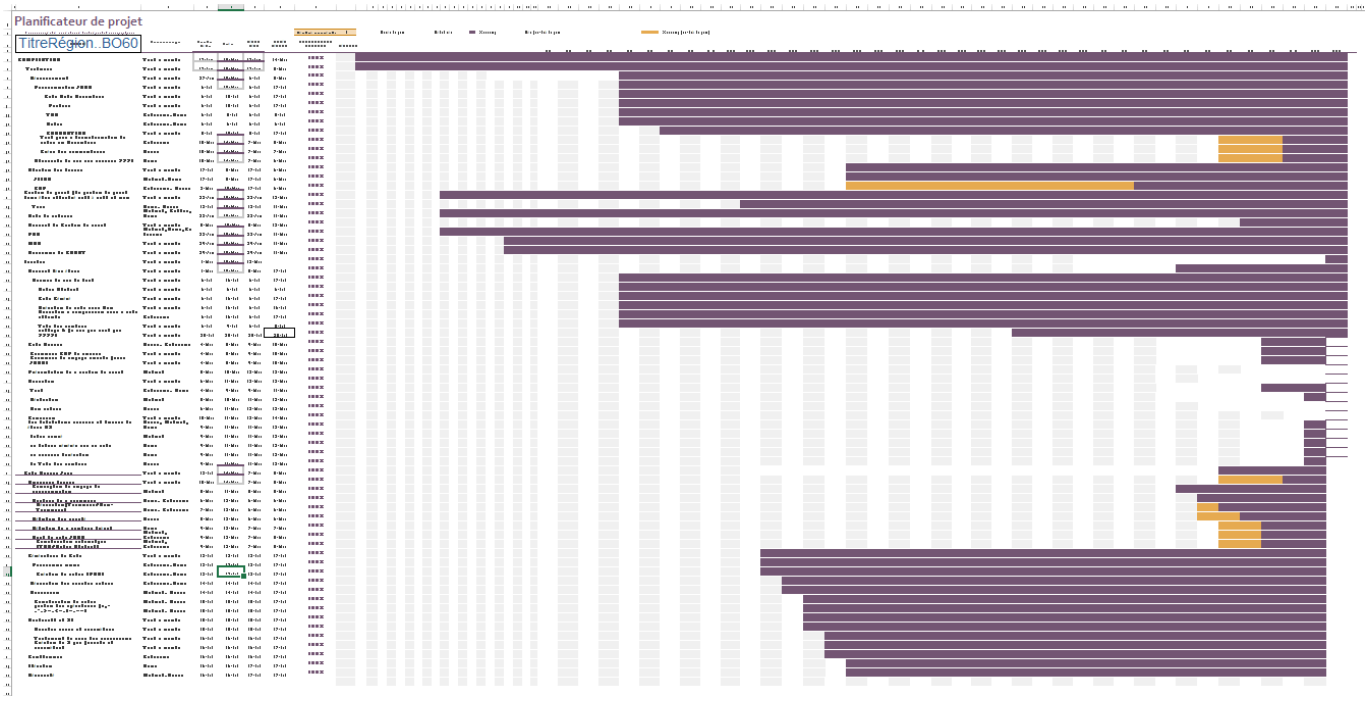


Figure 2 : Diagramme de Gantt du projet COMPILEXXL

Le diagramme de Gantt permet de visualiser la planification d'un projet dans le temps. Dans le cadre du projet COMPILEXXL, il a été construit à partir du WBS à l'aide d'Excel. Il reprend les différentes tâches, leurs durées et leurs dates de réalisation.

Chaque tâche est représentée par une barre sur une chronologie, ce qui facilite la visualisation de la séquence et de la durée des différentes étapes du projet. La planification a notamment pris en compte le fait que certaines étapes ne peuvent être réalisées qu'après d'autres (par exemple, la génération de code dépend de la construction de l'arbre abstrait et de la TDS).

Le diagramme de Gantt nous a permis de coordonner le travail des membres de l'équipe et de suivre l'avancement du projet en comparant les dates prévus et les dates réelles.

5. Tableau Trello

Nous avons utilisé l'outil de collaboration Trello pour assurer le suivi opérationnel et quotidien du projet. Deux tableaux Trello ont été créés, chacun correspondant à une phase (sprint) du projet, ce qui nous a permis de structurer le travail de manière progressive et organisée.

Chaque tableau est construit selon le principe du Kanban, avec plusieurs colonnes représentant l'avancement des tâches : « À faire », « En cours », « Vérification » et « Terminé ».

Les tâches sont représentées par des cartes. Lorsqu'un membre de l'équipe commence une tâche, il déplace la carte vers la colonne « En cours », permettant ainsi à toute l'équipe de suivre sa progression en temps réel. Une fois la tâche terminée, elle est déplacée vers la colonne « Vérification », où les autres membres de l'équipe peuvent examiner le travail réalisé. Après validation collective, la carte est déplacée vers la colonne « Terminé ».

Chaque carte contient des informations détaillées permettant un suivi précis :

- nom de la tâche,
- des dates de début et de fin,
- les membres assignés,
- des check-lists structurées,
- des étiquettes de couleur pour la classification des tâches.

Nous avons utilisé un système d'étiquettes pour distinguer les différentes parties du projet:

- Bleu : tâches liées à la partie de compilation
- Jaune : tâches liées à la gestion de projet
- Vert : rédaction du rapport et préparation de la présentation

Sprint 1

Vous pouvez consulter notre sprint 1 en accédant au tableau Trello via le lien suivant : https://trello.com/invite/b/698df54145fa7479b79d51dd/ATTI201f073ce7bfcc209aade66fc7a8987a3E5B1A01/sprint-1

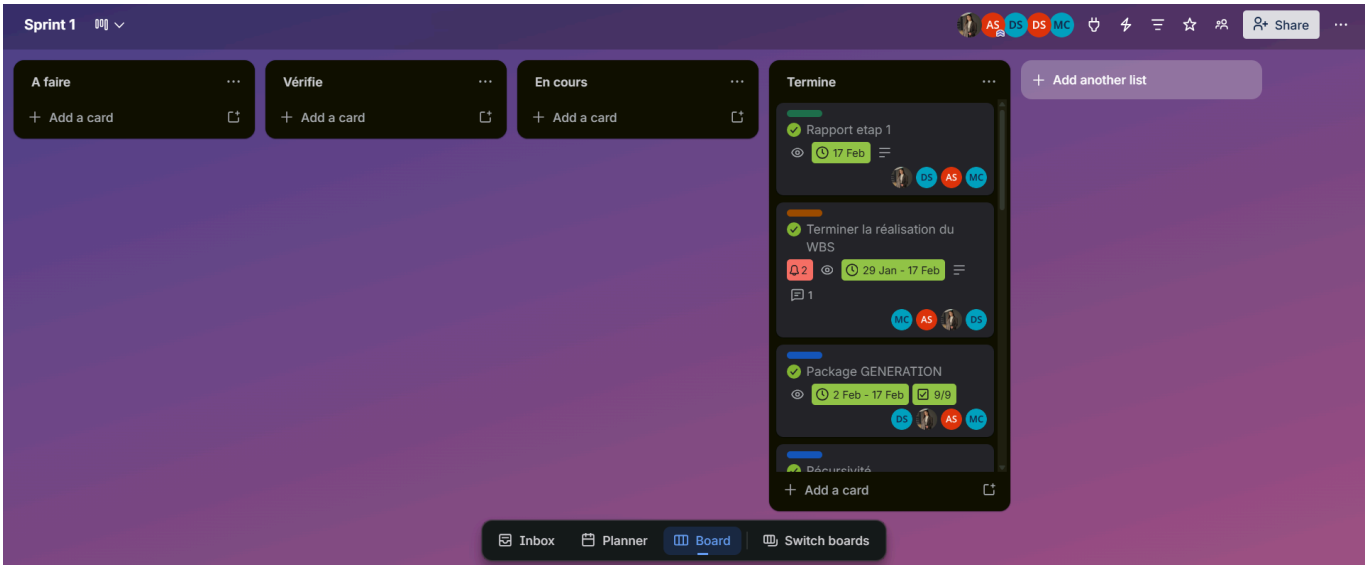


Figure 3 : Tableau Trello - Sprint 1

Sprint 2

Vous pouvez consulter notre sprint 2 en accédant au tableau Trello via le lien suivant : https://trello.com/invite/b/698df9ca1e173e87a797c983/ATTI57b7219e10bd23dc01414931500e97a1B7BBDEF1/sprint-2

Figure 4 : Tableau Trello - Sprint 2

II. PARTIE COMPILATION

1. Suite de tests

Exemple 1 : Programme minimal

```
vide main() {
}
```

Exemple 2 : Variables globales

```
entier i = 10;
entier j = 20;
entier k;
entier l;

vide main() {
}
```

Exemple 3 : Expression

```
entier x = 0;
entier a = 100;
entier b = 170;

vide main() {
   x = a * 2 + (b - 5) / 3;
}
```

Exemple 4 : Ecriture

```
entier x;

vide main() {
   x = lire() * 2 + (lire() - 5) / 3;
   ecrire(x);
}
```

Exemple 5 : Ecriture

```
entier a = 100;
entier b = 170;

vide main() {
   ecrire(a * 2 + (b - 5) / 3);
}
```

Exemple 6 : Variables locales et paramètres

```
entier a = 100;
entier c = 170;
```

```
entier f(entier a, entier b) {
   entier res;
   res = a * 2 + (b - 5) / 3;
   retourner res;
}

vide main() {
   ecrire(f(a, c));
}
```

Exemple 7 : Conditionnelle

```
entier a = 1;
entier b = 2;
entier x;

vide main() {
   si (a > b) {
       x = 1000;
   } sinon {
       x = 2000;
   }
}
```

Exemple 8 : Itération

```
entier i;

vide main() {
   i = 0;
   tantque (i < 6) {
       ecrire(i);
       i = i + 1;
   }
}
```

Exemple 9 : Récursivité

```
entier f(entier a) {
   si (a <= 0) {
       retourner 0;
   }
   retourner a + f(a - 1);
}

vide main() {
   ecrire(f(6));
}
```

2. Grammaire

2.1. La grammaire CUP de notre langage

La signification des abréviations utilisées pour les terminaux du langage :

- ENTIER : type entier (int)
- VIDE : type vide (void)
- SI : condition (if)
- SINON : alternative (else)
- TANTQUE : boucle (while)
- RETOURNER : instruction de retour (return)
- ECRIRE : affichage (print)
- LIRE : lecture d'une valeur (read)
- PLUS : addition +
- MOINS : soustraction -
- MUL : multiplication *
- DIV : division /
- AFF : affectation =
- SUP : supérieur >
- INF : inférieur <
- SUPE : supérieur ou égal >=
- INFE : inférieur ou égal <=
- EG : égal ==
- DIF : différent !=
- PO : parenthèse ouvrante (
- PF : parenthèse fermante)
- AO : accolade ouvrante {
- AF : accolade fermante }
- PV : point-virgule ;
- VIRG : virgule ,
- NOMBRE : constante entière
- IDF : identifiant (nom de variable ou de fonction)

```
package generated.fr.l3.miashs;

/* GRAMMAIRE*/
/* 1. terminaux */
terminal ENTIER, VIDE;
terminal SI, SINON, TANTQUE, RETOURNER, ECRIRE, LIRE;

terminal PLUS, MOINS, MUL, DIV;
terminal AFF;
terminal SUP, INF, SUPE, INFE, EG, DIF;

terminal PO, PF, AO, AF, PV, VIRG;

terminal Integer NOMBRE;
terminal String IDF;

/* 2. non terminaux */
non terminal programme;
non terminal liste_definitions;
non terminal definition;
non terminal suite_definition_entier;

non terminal liste_parametres_opt;
non terminal liste_parametres;
non terminal parametre;

non terminal bloc;
non terminal declarations_locales_opt;
non terminal declarations_locales;
non terminal declaration_locale;

non terminal liste_instructions_opt;
```

```
non terminal liste_instructions;
non terminal instruction;

non terminal affectation;
non terminal ecriture;
non terminal retour;
non terminal alternative;
non terminal repetitive;
non terminal appel;

non terminal liste_arguments_opt;
non terminal liste_arguments;

non terminal condition;
non terminal operateur_comparaison;

non terminal expression;
non terminal terme;
non terminal facteur;

/* priorités */
precedence left PLUS, MOINS;
precedence left MUL, DIV;

/* 3. axiome / start */
start with programme;

/* 4. REGLES */
programme ::= liste_definitions
 ;

liste_definitions ::= liste_definitions definition
 | definition
 ;

/* définitions globales */
definition ::= ENTIER IDF suite_definition_entier
 | VIDE IDF PO liste_parametres_opt PF bloc
 ;

/* après ENTIER IDF, on distingue variable globale ou fonction */
suite_definition_entier ::= PV
 | AFF expression PV
 | PO liste_parametres_opt PF bloc
 ;

/* paramètres */
liste_parametres_opt ::= liste_parametres
 |
 ;

liste_parametres ::= liste_parametres VIRG parametre
 | parametre
 ;

parametre ::= ENTIER IDF
 ;

/* bloc */
bloc ::= AO declarations_locales_opt liste_instructions_opt AF
 ;
```

```
/* variables locales */
declarations_locales_opt ::= declarations_locales
 |
 ;

declarations_locales ::= declarations_locales declaration_locale
 | declaration_locale
 ;

declaration_locale ::= ENTIER IDF PV
 ;

/* instructions */
liste_instructions_opt ::= liste_instructions
 |
 ;

liste_instructions ::= liste_instructions instruction
 | instruction
 ;

instruction ::= affectation PV
 | ecriture PV
 | appel PV
 | retour PV
 | alternative
 | repetitive
 ;

/* affectation */
affectation ::= IDF AFF expression
 ;

/* écriture */
ecriture ::= ECRIRE PO expression PF
 ;

/* retour */
retour ::= RETOURNER expression
 ;

/* alternative */
alternative ::= SI PO condition PF bloc
 | SI PO condition PF bloc SINON bloc
 ;

/* répétitive */
repetitive ::= TANTQUE PO condition PF bloc
 ;

/* appel */
appel ::= IDF PO liste_arguments_opt PF
 ;

liste_arguments_opt ::= liste_arguments
 |
 ;

liste_arguments ::= liste_arguments VIRG expression
 | expression
 ;
```

```
/* condition */
condition ::= expression operateur_comparaison expression
 ;

operateur_comparaison ::= SUP
 | INF
 | SUPE
 | INFE
 | EG
 | DIF
 ;

/* expressions */
expression ::= expression PLUS terme
 | expression MOINS terme
 | terme
 ;

terme ::= terme MUL facteur
 | terme DIV facteur
 | facteur
 ;

facteur ::= NOMBRE
 | IDF
 | appel
 | LIRE PO PF
 | PO expression PF
 ;
```

2.2. La grammaire enrichie du langage

La grammaire de notre langage contient désormais du code Java qui est exécuté lors de l'analyse syntaxique du programme (voir Annexe 5). Grâce à ce code, le parser construit directement l'arbre abstrait du programme. En même temps, il remplit la table des symboles en ajoutant les variables globales, les fonctions, les paramètres et les variables locales. Pour gérer la table des symboles, nous avons utilisé la classe *ConstructeurTds* (voir Annexe 6). Cette classe permet d'ajouter les différents symboles dans la TDS lorsqu'ils sont reconnus dans la grammaire. Donc, pendant l'analyse du programme, le parser construit à la fois l'arbre du programme et la table des symboles.

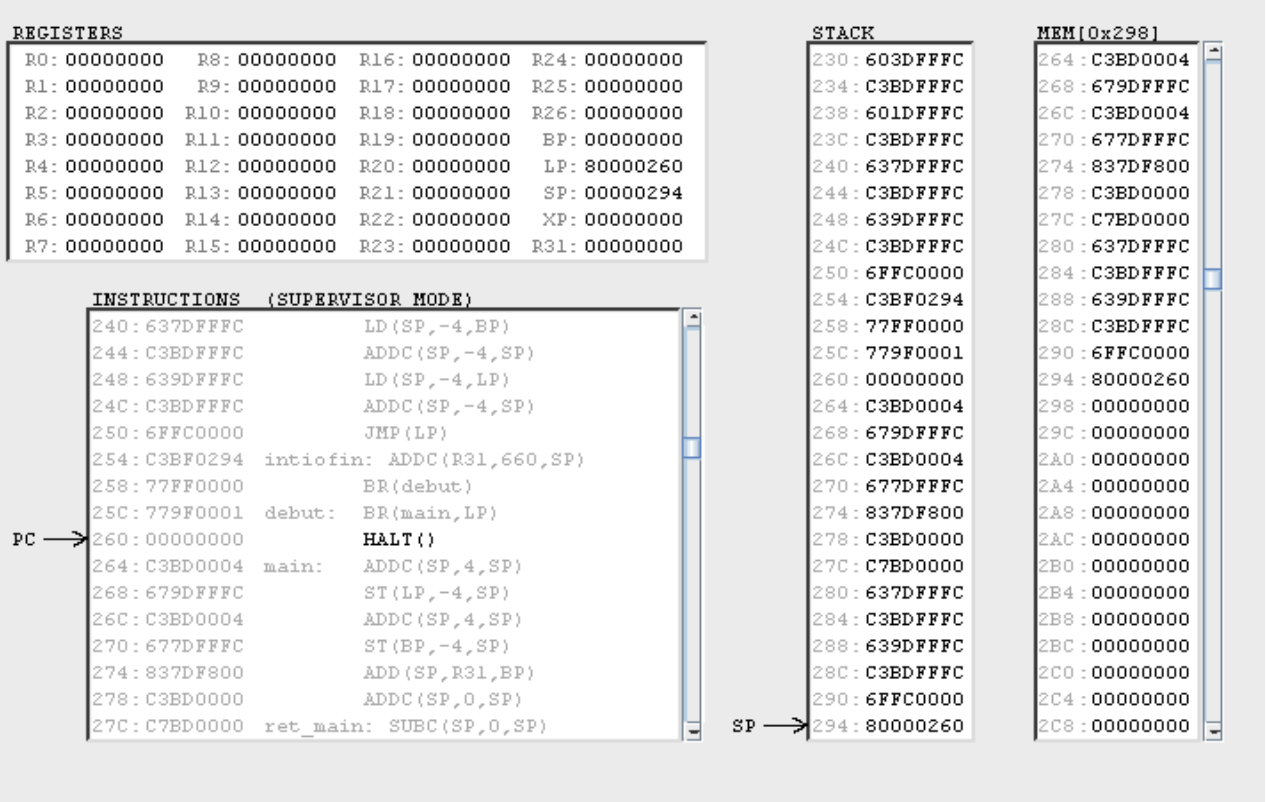
3. Évaluation

Dans cette deuxième étape, nous réutilisons les mêmes programmes de test que dans l'étape 1. La différence principale est que l'arbre abstrait et la table des symboles sont désormais construits automatiquement par l'analyseur syntaxique (CUP). Ainsi, pour éviter les redondances, les explications détaillées du fonctionnement et de la génération de code ne seront pas reprises ici. Elles sont présentées dans le rapport de l'étape 1. Nous nous concentrons donc uniquement sur la validation de l'analyse syntaxique, de la construction de l'arbre abstrait et de la TDS.

Exemple 1 : Programme minimal

Le programme est correctement analysé par le parseur. L'arbre abstrait contient uniquement la fonction main, ce qui correspond au cas minimal attendu. La table des symboles contient uniquement la fonction main, correctement identifiée. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement du programme et valide le parsing.

Le résultat de notre programme peut être consulté en Annexe 7.
Exécution avec BSim :



Exemple 2 : Variables globales

Le programme est correctement analysé. La table des symboles contient les variables globales avec leurs valeurs initiales, y compris les variables non initialisées. L'arbre abstrait reste minimal, ce qui est cohérent avec l'absence d'instructions dans main. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme la bonne gestion des variables globales.

Le résultat de notre programme peut être consulté en Annexe 8.
Exécution avec BSim :

```
REGISTERS
R0: 00000000   R8: 00000000  R16: 00000000  R24: 00000000
R1: 00000000   R9: 00000000  R17: 00000000  R25: 00000000
R2: 00000000  R10: 00000000  R18: 00000000  R26: 00000000
R3: 00000000  R11: 00000000  R19: 00000000   BP: 00000000
R4: 00000000  R12: 00000000  R20: 00000000   LP: 80000270
R5: 00000000  R13: 00000000  R21: 00000000   SP: 000002A4
R6: 00000000  R14: 00000000  R22: 00000000   XP: 00000000
R7: 00000000  R15: 00000000  R23: 00000000  R31: 00000000

INSTRUCTIONS  (SUPERVISOR MODE)
    250:6FFC0000          JMP(LP)
    254:C3BF02A4  intiofin: ADDC(R31,676,SP)
    258:77FF0004          BR(debut)
    25C:0000000A  i:      illop
    260:00000014  j:      illop
    264:00000000  k:      HALT()
    268:00000000  l:      HALT()
    26C:779F0001  debut:  BR(main,LP)
PC ->270:00000000         HALT()
    274:C3BD0004  main:   ADDC(SP,4,SP)
    278:679DFFFC          ST(LP,-4,SP)
    27C:C3BD0004          ADDC(SP,4,SP)
    280:677DFFFC          ST(BP,-4,SP)
    284:837DF800          ADD(SP,R31,BP)
    288:C3BD0000          ADDC(SP,0,SP)
    28C:C7BD0000  ret_main: SUBC(SP,0,SP)

STACK
240:637DFFFC
244:C3BDFFFC
248:639DFFFC
24C:C3BDFFFC
250:6FFC0000
254:C3BF02A4
258:77FF0004
25C:0000000A
260:00000014
264:00000000
268:00000000
26C:779F0001
270:00000000
274:C3BD0004
278:679DFFFC
27C:C3BD0004
280:677DFFFC
284:837DF800
288:C3BD0000
28C:C7BD0000
290:637DFFFC
294:C3BDFFFC
298:639DFFFC
29C:C3BDFFFC
2A0:6FFC0000
SP ->2A4:80000270

MEM[0x2a8]
274:C3BD0004
278:679DFFFC
27C:C3BD0004
280:677DFFFC
284:837DF800
288:C3BD0000
28C:C7BD0000
290:637DFFFC
294:C3BDFFFC
298:639DFFFC
29C:C3BDFFFC
2A0:6FFC0000
2A4:80000270
2A8:00000000
2AC:00000000
2B0:00000000
2B4:00000000
2B8:00000000
2BC:00000000
2C0:00000000
2C4:00000000
2C8:00000000
2CC:00000000
2D0:00000000
2D4:00000000
2D8:00000000
```

Exemple 3 : Expression

Le programme est correctement analysé par le parseur. L'arbre abstrait reflète correctement la structure de l'expression et respecte la priorité des opérations (multiplication/division avant addition). La table des symboles contient les variables globales utilisées. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme que le calcul est correct et que le parsing de l'expression fonctionne correctement.

Le résultat de notre programme peut être consulté en Annexe 9.

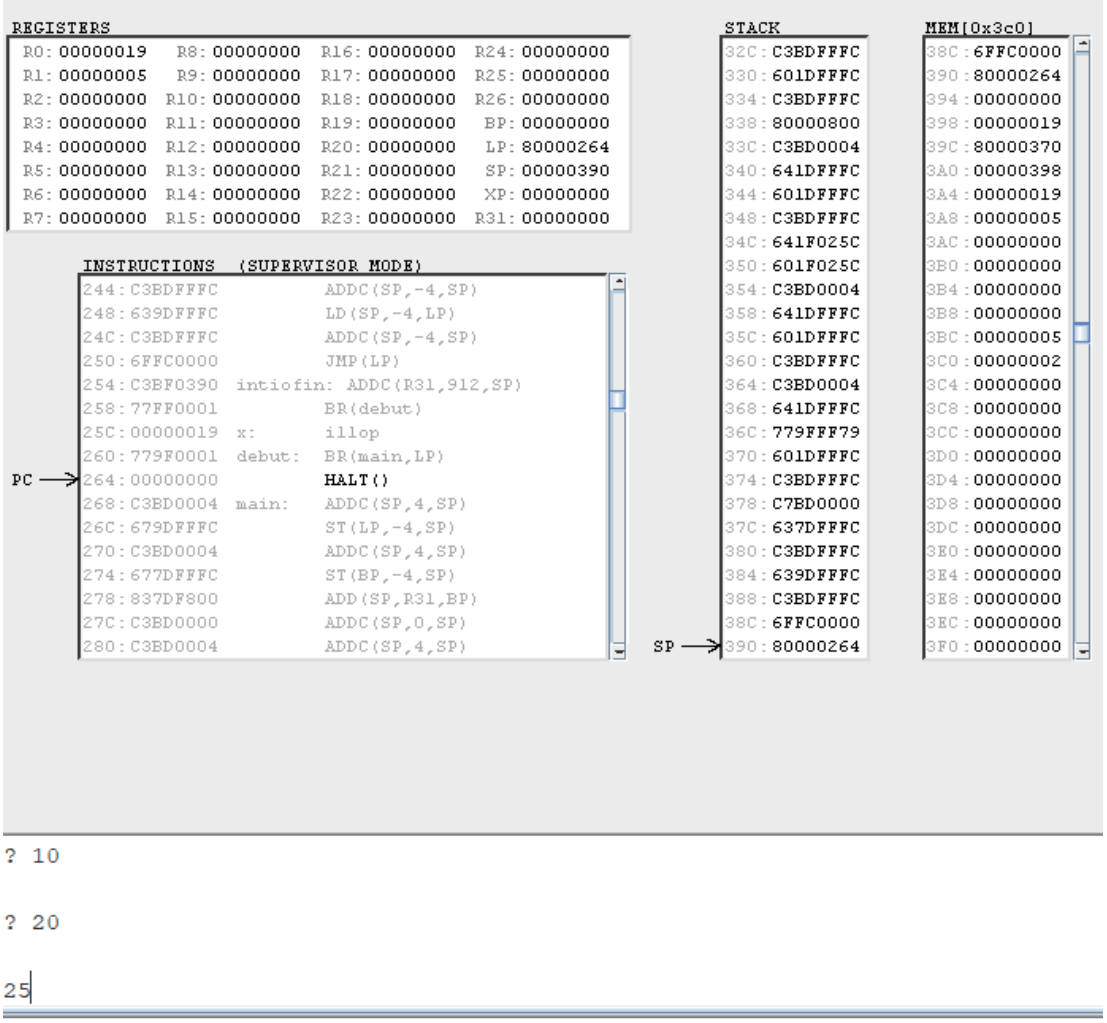
Exécution avec BSim :

```
REGISTERS
R0: 000000FF   R8: 00000000  R16: 00000000  R24: 00000000
R1: 00000037   R9: 00000000  R17: 00000000  R25: 00000000
R2: 00000000  R10: 00000000  R18: 00000000  R26: 00000000
R3: 00000000  R11: 00000000  R19: 00000000   BP: 00000000
R4: 00000000  R12: 00000000  R20: 00000000   LP: 8000026C
R5: 00000000  R13: 00000000  R21: 00000000   SP: 00000358
R6: 00000000  R14: 00000000  R22: 00000000   XP: 00000000
R7: 00000000  R15: 00000000  R23: 00000000  R31: 00000000

INSTRUCTIONS  (SUPERVISOR MODE)
    24C:C3BDFFFC          ADDC(SP,-4,SP)
    250:6FFC0000          JMP(LP)
    254:C3BF0358  intiofin: ADDC(R31,856,SP)
    258:77FF0003          BR(debut)
    25C:000000FF  x:      illop
    260:00000064  a:      illop
    264:000000AA  b:      illop
    268:779F0001  debut:  BR(main,LP)
PC ->26C:00000000         HALT()
    270:C3BD0004  main:   ADDC(SP,4,SP)
    274:679DFFFC          ST(LP,-4,SP)
    278:C3BD0004          ADDC(SP,4,SP)
    27C:677DFFFC          ST(BP,-4,SP)
    280:837DF800          ADD(SP,R31,BP)
    284:C3BD0000          ADDC(SP,0,SP)
    288:601F0260          LD(R31,608,R0)

STACK
2F4:C3BD0004
2F8:641DFFFC
2FC:603DFFFC
300:C3BDFFFC
304:601DFFFC
308:C3BDFFFC
30C:8C000800
310:C3BD0004
314:641DFFFC
318:603DFFFC
31C:C3BDFFFC
320:601DFFFC
324:C3BDFFFC
328:80000800
32C:C3BD0004
330:641DFFFC
334:601DFFFC
338:C3BDFFFC
33C:641F025C
340:C7BD0000
344:637DFFFC
348:C3BDFFFC
34C:639DFFFC
350:C3BDFFFC
354:6FFC0000
SP ->358:8000026C

MEM[0x25c]
228:605DFFFC
22C:C3BDFFFC
230:603DFFFC
234:C3BDFFFC
238:601DFFFC
23C:C3BDFFFC
240:637DFFFC
244:C3BDFFFC
248:639DFFFC
24C:C3BDFFFC
250:6FFC0000
254:C3BF0358
258:77FF0003
25C:000000FF
260:00000064
264:000000AA
268:779F0001
26C:00000000
270:C3BD0004
274:679DFFFC
278:C3BD0004
27C:677DFFFC
280:837DF800
284:C3BD0000
288:601F0260
28C:C3BD0004
```

Exemple 4 : Ecriture

Le programme est correctement analysé. L'arbre abstrait contient une affectation suivie d'une instruction d'écriture, ce qui correspond à la structure attendue. La table des symboles contient la variable globale utilisée. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement des entrées/sorties et du parsing.

Le résultat de notre programme peut être consulté en Annexe 10.
Exécution avec BSim :



Exemple 5 : Ecriture

Le programme est correctement analysé. L'arbre abstrait contient uniquement une instruction d'écriture avec une expression complexe. La structure de l'expression est correctement représentée dans l'arbre abstrait. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme que l'expression est correctement évaluée et affichée.

Le résultat de notre programme peut être consulté en Annexe 11.
Exécution avec BSim :

```
REGISTERS
R0: 000000FF   R8: 00000000  R16: 00000000  R24: 00000000
R1: 00000037   R9: 00000000  R17: 00000000  R25: 00000000
R2: 00000000  R10: 00000000  R18: 00000000  R26: 00000000
R3: 00000000  R11: 00000000  R19: 00000000   BP: 00000000
R4: 00000000  R12: 00000000  R20: 00000000   LP: 80000268
R5: 00000000  R13: 00000000  R21: 00000000   SP: 00000364
R6: 00000000  R14: 00000000  R22: 00000000   XP: 00000000
R7: 00000000  R15: 00000000  R23: 00000000  R31: 00000000

INSTRUCTIONS  (SUPERVISOR MODE)
248:639DFFFC          LD(SP,-4,LP)
24C:C3BDFFFC          ADDC(SP,-4,SP)
250:6FFC0000          JMP(LP)
254:C3BF0364  intiofin: ADDC(R31,868,SP)
258:77FF0002          BR(debut)
25C:00000064  a:      illop
260:000000AA  b:      illop
264:779F0001  debut:  BR(main,LP)
PC -> 268:00000000    HALT()
26C:C3BD0004  main:   ADDC(SP,4,SP)
270:679DFFFC          ST(LP,-4,SP)
274:C3BD0004          ADDC(SP,4,SP)
278:677DFFFC          ST(BP,-4,SP)
27C:837DF800          ADD(SP,R31,BP)
280:C3BD0000          ADDC(SP,0,SP)
284:601F025C          LD(R31,604,R0)

STACK
300:601DFFFC
304:C3BDFFFC
308:8C000800
30C:C3BD0004
310:641DFFFC
314:603DFFFC
318:C3BDFFFC
31C:601DFFFC
320:C3BDFFFC
324:80000800
328:C3BD0004
32C:641DFFFC
330:601DFFFC
334:C3BDFFFC
338:C3BD0004
33C:641DFFFC
340:779FFF84
344:601DFFFC
348:C3BDFFFC
34C:C7BD0000
350:637DFFFC
354:C3BDFFFC
358:639DFFFC
35C:C3BDFFFC
360:6FFC0000
SP -> 364:80000268

MEM[0x398]
364:80000268
368:00000000
36C:000000FF
370:80000344
374:0000036C
378:000000FF
37C:00000037
380:00000000
384:00000000
388:00000000
38C:00000000
390:00000005
394:00000005
398:00000002
39C:00000000
3A0:00000000
3A4:00000000
3A8:00000000
3AC:00000000
3B0:00000000
3B4:00000000
3B8:00000000
3BC:00000000
3C0:00000000
3C4:00000000
3C8:00000000

255
```

Exemple 6 : Variables locales et paramètres

Le programme est correctement analysé. La table des symboles montre une gestion correcte des portées : les paramètres et variables locales sont associés à leur fonction (notation a@f, b@f, res@f). L'arbre abstrait représente correctement les fonctions, l'affectation et le retour. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement et du parsing.

Le résultat de notre programme peut être consulté en Annexe 12.

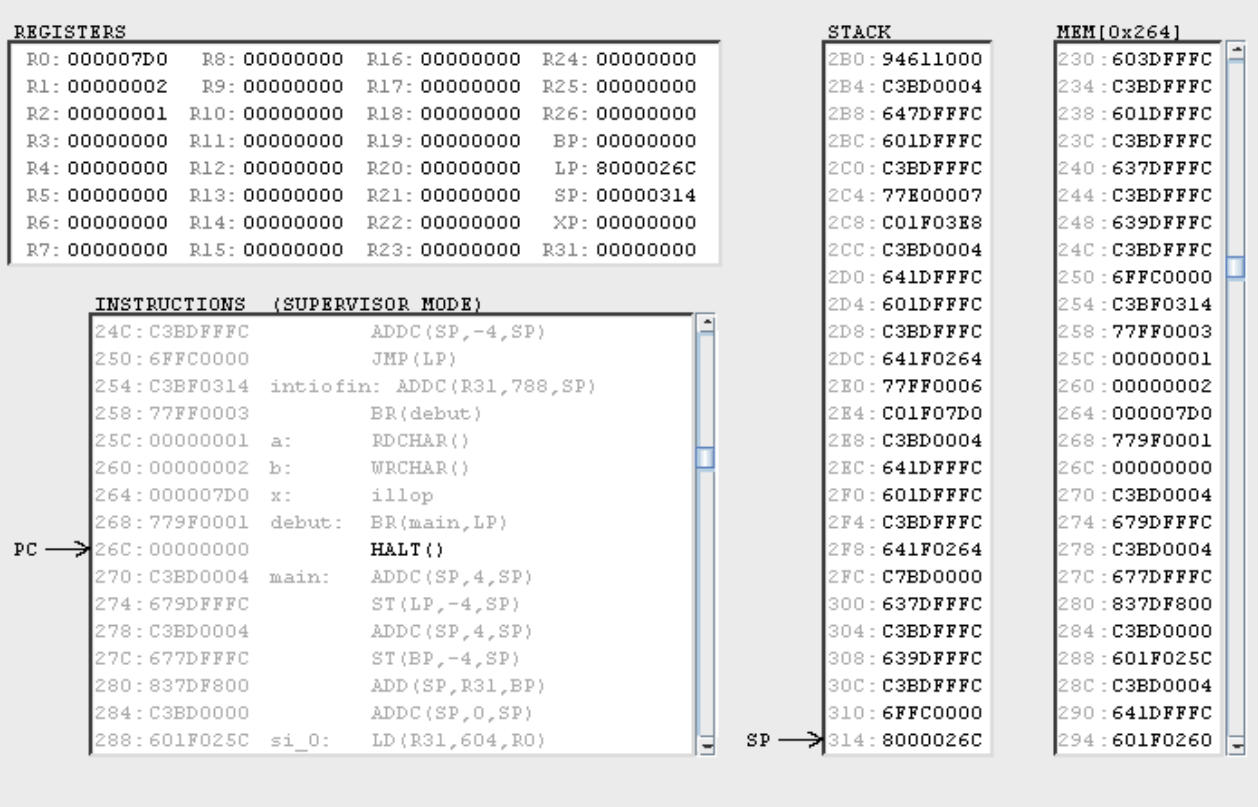
Exécution avec BSim :

```
REGISTERS
R0: 000000FF   R8: 00000000  R16: 00000000  R24: 00000000
R1: 00000037   R9: 00000000  R17: 00000000  R25: 00000000
R2: 00000000  R10: 00000000  R18: 00000000  R26: 00000000
R3: 00000000  R11: 00000000  R19: 00000000   BP: 00000000
R4: 00000000  R12: 00000000  R20: 00000000   LP: 80000268
R5: 00000000  R13: 00000000  R21: 00000000   SP: 000003E0
R6: 00000000  R14: 00000000  R22: 00000000   XP: 00000000
R7: 00000000  R15: 00000000  R23: 00000000  R31: 00000000

INSTRUCTIONS  (SUPERVISOR MODE)
248:639DFFFC          LD(SP,-4,LP)
24C:C3BDFFFC          ADDC(SP,-4,SP)
250:6FFC0000          JMP(LP)
254:C3BF03E0  intiofin: ADDC(R31,992,SP)
258:77FF0002          BR(debut)
25C:00000064  a:      illop
260:000000AA  c:      illop
264:779F0042  debut:  BR(main,LP)
PC -> 268:00000000    HALT()
26C:C3BD0004  f:      ADDC(SP,4,SP)
270:679DFFFC          ST(LP,-4,SP)
274:C3BD0004          ADDC(SP,4,SP)
278:677DFFFC          ST(BP,-4,SP)
27C:837DF800          ADD(SP,R31,BP)
280:C3BD0004          ADDC(SP,4,SP)
284:601BFFF0          LD(BP,-16,R0)

STACK
37C:677DFFFC
380:837DF800
384:C3BD0000
388:C3BD0004
38C:601F025C
390:C3BD0004
394:641DFFFC
398:601F0260
39C:C3BD0004
3A0:641DFFFC
3A4:779FFFB1
3A8:C7BD0008
3AC:601DFFFC
3B0:C3BDFFFC
3B4:C3BD0004
3B8:641DFFFC
3BC:779FFF65
3C0:601DFFFC
3C4:C3BDFFFC
3C8:C7BD0000
3CC:637DFFFC
3D0:C3BDFFFC
3D4:639DFFFC
3D8:C3BDFFFC
3DC:6FFC0000
SP -> 3E0:80000268

MEM[0x414]
3E0:80000268
3E4:00000000
3E8:000000FF
3EC:800003C0
3F0:000003E8
3F4:000000FF
3F8:00000037
3FC:00000000
400:00000000
404:00000000
408:00000000
40C:00000005
410:00000005
414:00000002
418:00000000
41C:00000000
420:00000000
424:00000000
428:00000000
42C:00000000
430:00000000
434:00000000
438:00000000
43C:00000000
440:00000000
444:00000000

255
```

Exemple 7 : Conditionnelle

Le programme est correctement analysé. L'arbre abstrait contient une structure conditionnelle avec un bloc *alors* et un bloc *sinon*. La table des symboles contient les variables globales utilisées. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement de la conditionnelle et du parsing.

Le résultat de notre programme peut être consulté en Annexe 13.
Exécution avec BSim :



Exemple 8 : Itération

Le programme est correctement analysé. L'arbre abstrait contient une boucle *tant que* avec une condition et un bloc d'instructions, ce qui correspond à la structure attendue. La variable globale est correctement gérée dans la table des symboles. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement de la boucle et du parsing.

Le résultat de notre programme peut être consulté en Annexe 14.
Exécution avec BSim :

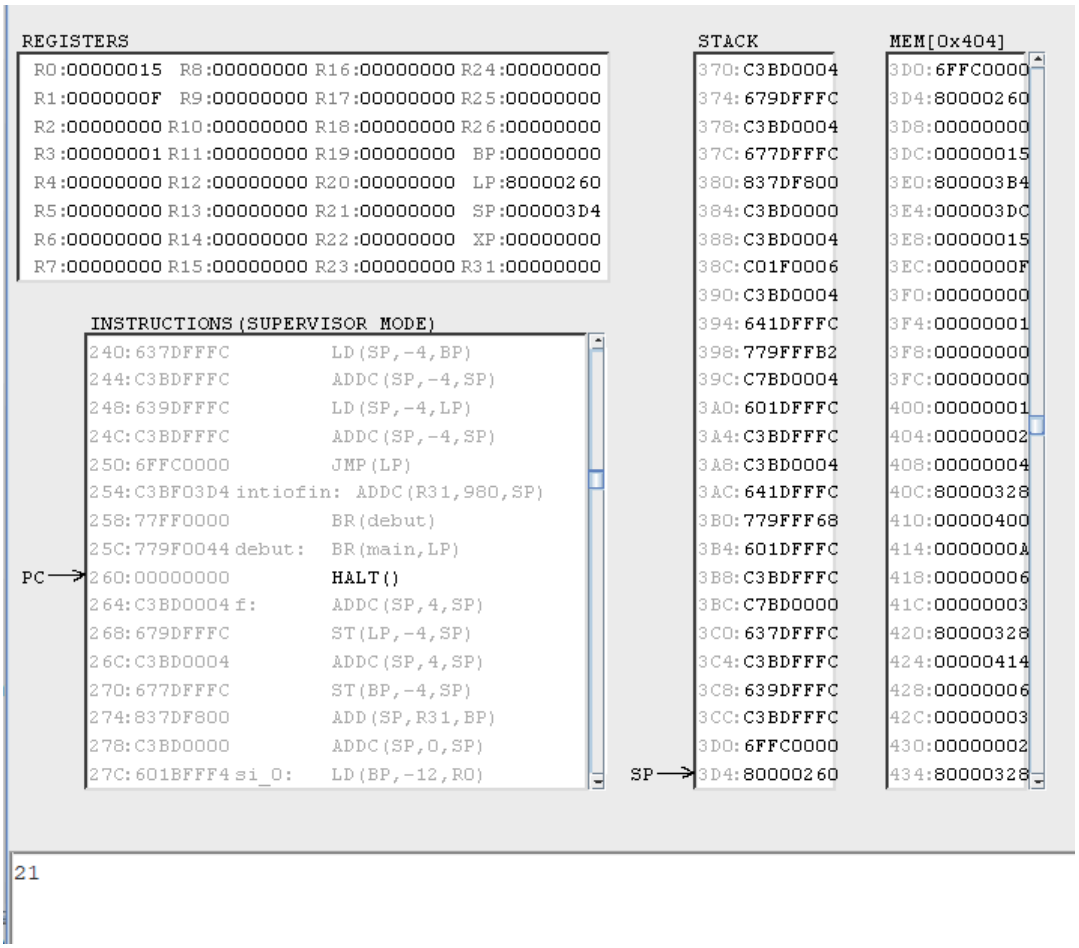
```
REGISTERS
R0:00000000  R8:00000000 R16:00000000 R24:00000000
R1:00000006  R9:00000000 R17:00000000 R25:00000000
R2:00000006 R10:00000000 R18:00000000 R26:00000000
R3:00000000 R11:00000000 R19:00000000  BP:00000000
R4:00000000 R12:00000000 R20:00000000  LP:80000264
R5:00000000 R13:00000000 R21:00000000  SP:0000035C
R6:00000000 R14:00000000 R22:00000000  XP:00000000
R7:00000000 R15:00000000 R23:00000000 R31:00000000

INSTRUCTIONS (SUPERVISOR MODE)
    244:C3BDFFFC          ADDC(SP,-4,SP)
    248:639DFFFC          LD(SP,-4,LP)
    24C:C3BDFFFC          ADDC(SP,-4,SP)
    250:6FFC0000          JMP(LP)
    254:C3BF035C intiofin: ADDC(R31,860,SP)
    258:77FF0001          BR(debut)
    25C:00000006 i:       RANDOM()
    260:779F0001 debut:   BR(main,LP)
PC→ 264:00000000          HALT()
    268:C3BD0004 main:    ADDC(SP,4,SP)
    26C:679DFFFC          ST(LP,-4,SP)
    270:C3BD0004          ADDC(SP,4,SP)
    274:677DFFFC          ST(BP,-4,SP)
    278:837DF800          ADD(SP,R31,BP)
    27C:C3BD0000          ADDC(SP,0,SP)
    280:C01F0000          ADDC(R31,0,R0)

STACK
2F8:601DFFFC
2FC:C3BDFFFC
300:601F025C
304:C3BD0004
308:641DFFFC
30C:C01F0001
310:C3BD0004
314:641DFFFC
318:603DFFFC
31C:C3BDFFFC
320:601DFFFC
324:C3BDFFFC
328:80000800
32C:C3BD0004
330:641DFFFC
334:601DFFFC
338:C3BDFFFC
33C:641F025C
340:77FFFFD5
344:C7BD0000
348:637DFFFC
34C:C3BDFFFC
350:639DFFFC
354:C3BDFFFC
358:6FFC0000
SP→ 35C:80000264

MEM[0x364]
330:641DFFFC
334:601DFFFC
338:C3BDFFFC
33C:641F025C
340:77FFFFD5
344:C7BD0000
348:637DFFFC
34C:C3BDFFFC
350:639DFFFC
358:6FFC0000
35C:80000264
360:00000000
364:00000000
368:00000006
36C:00000364
370:00000005
374:00000006
378:00000005
37C:00000001
380:00000000
384:00000000
388:00000005
38C:00000000
390:00000000
394:00000000

0
1
2
3
4
```

```
2
3
4
5
```

Exemple 9 : Récursivité

Le programme est correctement analysé. L'arbre abstrait représente correctement la structure récursive de la fonction, avec un appel à elle-même. La table des symboles contient la fonction et son paramètre. Le code généré est conforme à celui obtenu lors de l'étape 1. L'exécution sous BSim confirme le bon fonctionnement de la récursivité et de la gestion de la pile.

Le résultat de notre programme peut être consulté en Annexe 15.
Exécution avec BSim :



Bilan critique

Les résultats obtenus démontrent que l'objectif principal de la deuxième étape a été atteint. Notre analyseur syntaxique peut reconnaître les programmes écrits dans notre langage, construire automatiquement un arbre abstrait et une table des symboles, puis réutiliser ces structures pour générer du code assembleur BETA correct.

Cependant, certaines limites subsistent. La gestion des erreurs pourrait être améliorée, notamment en fournissant des messages d'erreur plus précis. De plus, la vérification sémantique n'est pas encore complète (p. e., la vérification de l'existence des variables avant leur utilisation, la cohérence des appels de fonctions (nombre de paramètres)). Ces vérifications pourraient être davantage développées dans une version plus avancée du compilateur. Par ailleurs, le langage reste volontairement limité au cadre de ce sujet (type entier, instructions imposées, absence d'optimisation).

Le code généré est correct pour les exemples testés, mais il pourrait être optimisé dans une version plus avancée du compilateur. Malgré cela, les objectifs principaux ont été atteints et les résultats obtenus valident le bon fonctionnement global du compilateur sur le jeu de tests demandé.

4. Distribution binaire du compilateur

La version contient un fichier exécutable *compilateur_compilexxl.jar*, ainsi que des scripts de lancement (*run.bat* pour Windows et *run.sh* pour Linux) et un dossier *samples* avec des fichiers d'exemple.

Pour utiliser le compilateur, il est nécessaire d'avoir Java installé sur la machine. Il est recommandé d'utiliser une version 17 ou supérieure.

Utilisation sous Windows

L'utilisateur peut simplement ouvrir le fichier *run.bat* par un double-clic. Le programme demande ensuite de saisir le nom d'un fichier présent dans le dossier *samples*, par exemple : *ex1_programme_minimal.txt* . Le compilateur s'exécute automatiquement et affiche les résultats dans la console.

Utilisation en ligne de commande

Il est également possible de lancer le compilateur manuellement avec la commande suivante :

```
java -jar compilateur_compilexxl.jar samples/ex1_programme_minimal.txt
```

Utilisation sous Linux / WSL

Sous Linux ou WSL, il suffit d'exécuter la même commande dans le terminal :

```
java -jar compilateur_compilexxl.jar samples/ex1_programme_minimal.txt
```

Le dossier *samples* contient plusieurs programmes permettant de tester différentes fonctionnalités :

1. ex1_programme_minimal.txt
2. ex2_variables_globales.txt
3. ex3_expression.txt
4. ex4_ecriture.txt
5. ex5_ecriture.txt
6. ex6_variables_locales_et_parametres.txt
7. ex7_conditionnelle.txt
8. ex8_iteration.txt
9. ex9_recursivite.txt

CONCLUSION

Le projet COMPILEXXL nous a permis de concevoir et d'implémenter un compilateur fonctionnel pour un langage impératif simplifié. Nous avons pu réaliser différentes étapes de compilation, de l'analyse lexicale et syntaxique à la génération de code assembleur BETA.

Ce projet, à la fois technique et organisationnel, nous a permis de mettre en pratique des compétences issues de deux domaines : la compilation et la gestion de projet. Nous avons ainsi acquis une meilleure compréhension du fonctionnement interne du compilateur tout en développant nos compétences en travail d'équipe, en planification des tâches et en gestion de projet.

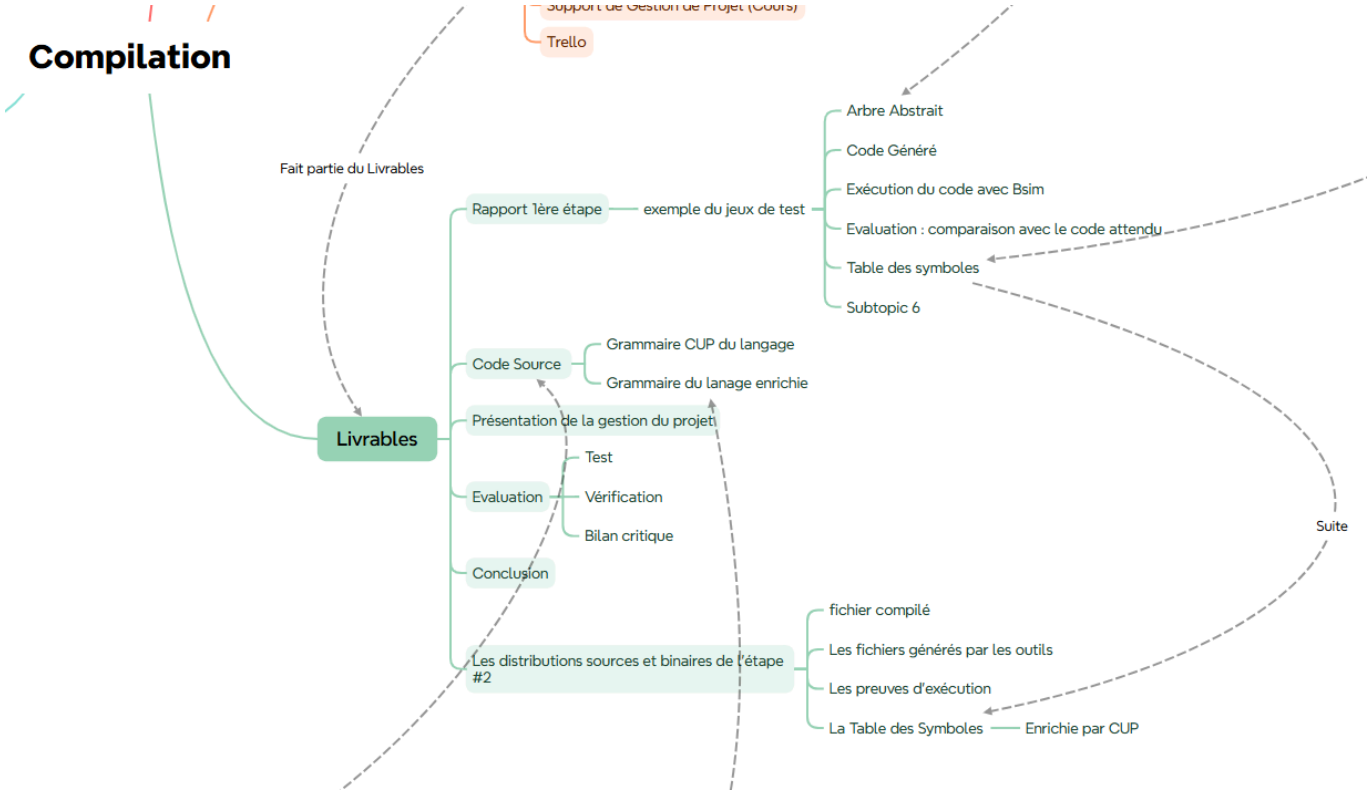
L'utilisation d'outils tels que PBS, WBS, le diagramme de Gantt, Trello et GitHub a permis de structurer efficacement le travail et d'assurer une bonne coordination entre les membres de l'équipe.

Les résultats obtenus montrent que le compilateur fonctionne correctement sur un ensemble de programmes de test couvrant les principales fonctionnalités du langage, notamment les expressions, les conditions, les boucles et la récursivité. Cependant, certaines limitations subsistent. La vérification sémantique demeure incomplète et le langage implémenté est volontairement simplifié. De plus, des améliorations pourraient être apportées, notamment en matière d'optimisation de la génération de code et de gestion des erreurs.

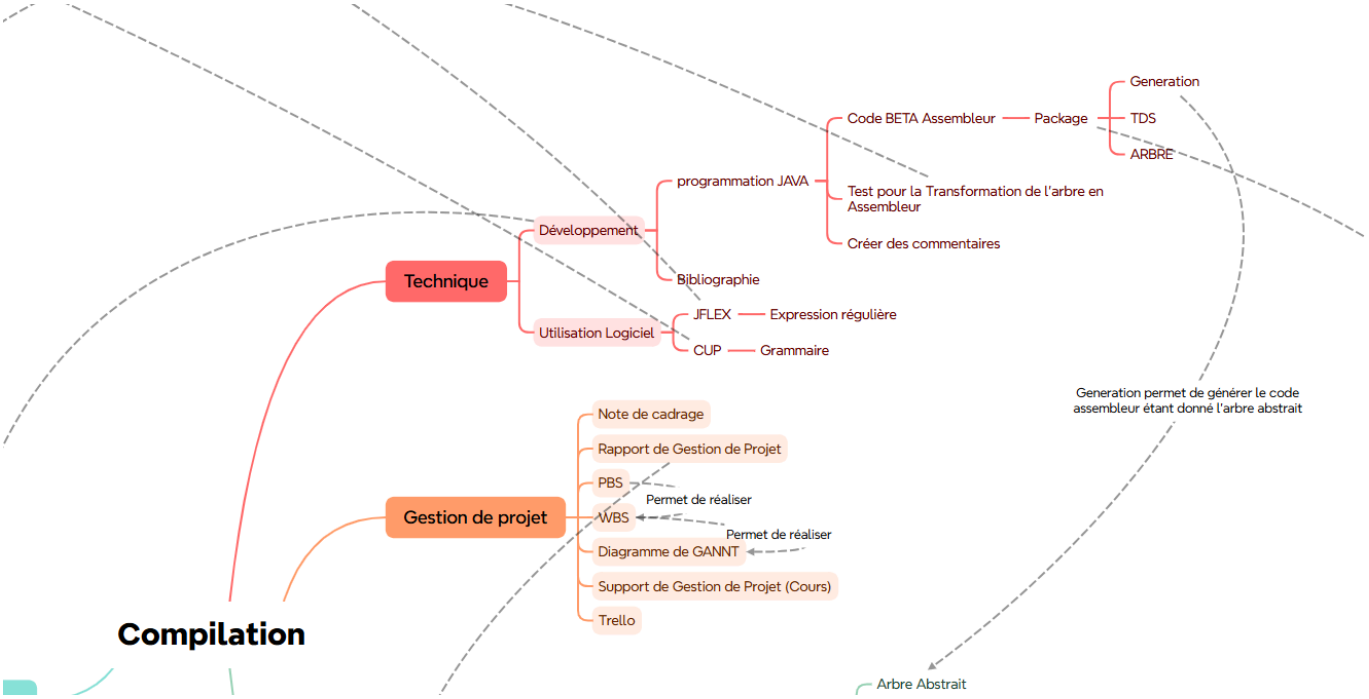
Malgré ces limites, les objectifs principaux du projet ont été atteints. Ce travail constitue une base solide pouvant être enrichie et améliorée dans le cadre de développements futurs.

ANNEXES

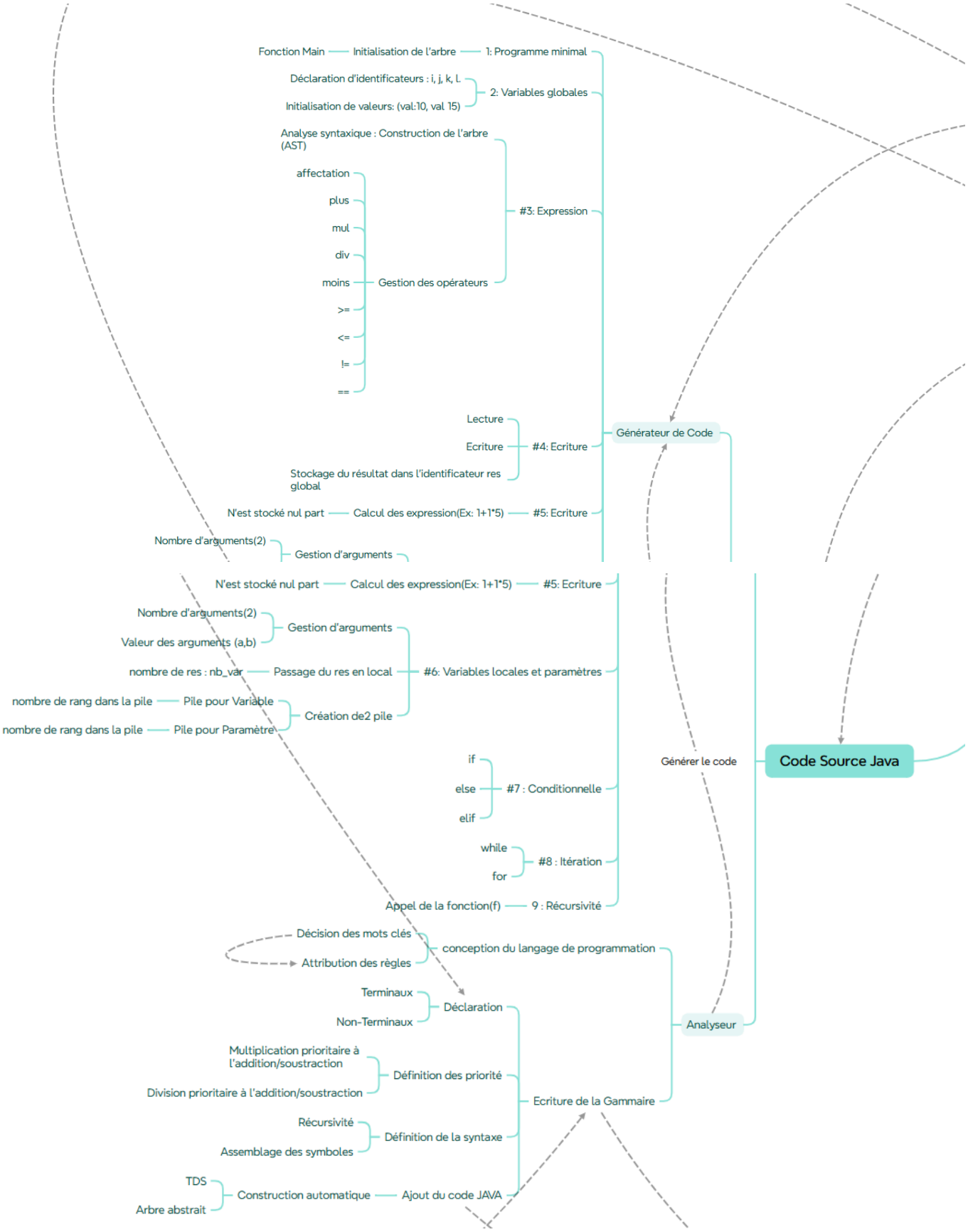
Annexe 1 : Les livrables



Annexe 2 : La gestion de projet et la partie technique



Annexe 3 : Le code source Java



Annexe 4 : WBS

Planificateur de projet

lectionnez une périodeà mettre en évidenceà droite. Une légende décrivant le graphique suit

Période à mettre en évi

	ACTIVITÉ	Qui s'en occupe	Date de début	Durée (J)	DÉBUT RÉEL	DURÉE RÉELLE	POURCENTAGE ACCOMPLI
5	COMPILATION	Tout le monde	13-Jan	19-Mar	13-Jan	14-Mar	100%
6	Technique	Tout le monde	13-Jan	19-Mar	13-Jan	8-Mar	100%
7	Développement	Tout le monde	27-Jan	19-Mar	6-Feb	8-Mar	100%
8	Programmation JAVA	Tout le monde	6-Feb	19-Mar	6-Feb	17-Feb	100%
9	Code Beta Assembleur	Tout le monde	6-Feb	15-Feb	6-Feb	17-Feb	100%
10	Package	Tout le monde	6-Feb	15-Feb	6-Feb	17-Feb	100%
11	TDS	Kateryna,Diana	6-Feb	8-Feb	6-Feb	8-Feb	100%
12	Arbre	Kateryna,Diana	6-Feb	6-Feb	6-Feb	6-Feb	100%
13	GENERATION	Tout le monde	8-Feb	15-Feb	8-Feb	17-Feb	100%
14	Test pour la transformation de l'arbre en Assembleur	Kateryna	10-Mar	14-Mar	7-Mar	8-Mar	100%
15	Créer des commentaires	Alyssa	10-Mar	14-Mar	7-Mar	7-Mar	100%
16	Bibliographie (je sais plus pourquoi ???)	Diana	10-Mar	14-Mar	7-Mar	6-Mar	100%
17	Utilisation des Logiciels	Tout le monde	17-Feb	5-Mar	17-Feb	6-Mar	100%
18	JFLEX	Mehmet,Diana	17-Feb	5-Mar	17-Feb	6-Mar	100%
19	CUP	Kateryna, Alyssa	3-Mar	10-Mar	17-Feb	6-Mar	100%
20	Gestion de projet (La gestion de projet dans etre effectue petit a patit et non d'une traite, voilà pourquoi les dates sont aussi espacé)	Tout le monde	22-Jan	19-Mar	22-Jan	12-Mar	100%
21	Trello	Diana, Alyssa	12-Feb	19-Mar	12-Feb	11-Mar	100%
22	Note de cadrage	Mehmet, Kathya, Diana	22-Jan	19-Mar	22-Jan	11-Mar	100%
23	Rapport de Gestion de projet	Tout le monde	8-Mar	10-Mar	8-Mar	12-Mar	100%
24	PBS	Mehmet,Diana,Kateryna	22-Jan	19-Mar	22-Jan	11-Mar	100%
25	WBS	Tout le monde	29-Jan	19-Mar	29-Jan	11-Mar	100%
26	Diagramme de GANNT	Tout le monde	29-Jan	19-Mar	29-Jan	11-Mar	100%
27	Livrables	Tout le monde	1-Mar	19-Mar	12-Mar		100%
28	Rapport 1ère étape	Tout le monde	1-Mar	19-Mar	5-Mar	17-Feb	100%
29	Exemple du jeux de test	Tout le monde	6-Feb	16-Feb	6-Feb	17-Feb	100%
30	Arbre Abstrait	Tout le monde	6-Feb	6-Feb	6-Feb	6-Feb	100%
31	Code Généré	Tout le monde	6-Feb	16-Feb	6-Feb	17-Feb	100%
32	Exécution du code avec Bsim	Tout le monde	6-Feb	16-Feb	6-Feb	16-Feb	100%
33	Evaluation : compairaison avec le code attendu	Kateryna	6-Feb	16-Feb	6-Feb	17-Feb	100%
34	Table des symboles	Tout le monde	6-Feb	9-Feb	6-Feb	8-Feb	100%
35	subtopic 6 (je sais plus c'est quoi ????)	Tout le monde	25-Feb	28-Feb	25-Feb	28-Feb	100%
36	Code Source	Alyssa, Kateryna	4-Mar	8-Mar	9-Mar	10-Mar	100%
37	Grammaire CUP du langage	Tout le monde	4-Mar	8-Mar	9-Mar	10-Mar	100%
38	Grammaire du langage enrichie (avec JAVA)	Tout le monde	4-Mar	8-Mar	9-Mar	10-Mar	100%
39	Présentation de la gestion du projet	Mehmet	8-Mar	10-Mar	12-Mar	13-Mar	100%
40	Evaluation	Tout le monde	6-Mar	11-Mar	12-Mar	13-Mar	100%
41	Test	Kateryna, Diana	4-Mar	9-Mar	9-Mar	11-Mar	100%
42	Vérification	Mehmet	5-Mar	10-Mar	11-Mar	12-Mar	100%
43	Bilan critique	Alyssa	6-Mar	11-Mar	12-Mar	13-Mar	100%
44	Conclusion	Tout le monde	10-Mar	11-Mar	13-Mar	14-Mar	100%
45	Les distributions sources et binaires de l'étape #2	Alyssa, Mehmet, Diana	9-Mar	11-Mar	11-Mar	12-Mar	100%
46	fichier compilé	Mehmet	9-Mar	11-Mar	11-Mar	12-Mar	100%
47	les fichiers générés par les outils	Diana	9-Mar	11-Mar	11-Mar	12-Mar	100%

Planificateur_de_projet

E61 | 2/13/2026

	A	B	C	D	E	F	G	H	I
49		La Table des symboles	Alyssa	9-Mar	11-Mar	11-Mar	12-Mar	100%	
50		Code Source Java	Tout le monde	13-Feb	14-Mar	7-Mar	8-Mar	100%	
51		Analyseur Lexicale	Tout le monde	10-Mar	14-Mar	7-Mar	8-Mar	100%	
52		Conception du langage de programmation	Mehmet	5-Mar	11-Mar	5-Mar	5-Mar	100%	
53		Ecriture de la grammaire	Diana, Kateryna	6-Mar	12-Mar	6-Mar	6-Mar	100%	
54		Déclaration(Terminaux/Non-Terminaux)	Diana, Kateryna	7-Mar	12-Mar	6-Mar	6-Mar	100%	
55		Définition des priorité	Alyssa	8-Mar	12-Mar	6-Mar	6-Mar	100%	
56		Définition de la syntaxe (règle)	Diana	9-Mar	13-Mar	7-Mar	7-Mar	100%	
57		Ajout du code JAVA	Mehmet, Kateryna	9-Mar	13-Mar	7-Mar	8-Mar	100%	
58		Construction automatique (TDS/Arbre Abstrait)	Mehmet, Kateryna	9-Mar	13-Mar	7-Mar	8-Mar	100%	
59		Générateur de Code	Tout le monde	13-Feb	13-Feb	13-Feb	17-Feb	100%	
60		Programme minimal	Kateryna,Diana	13-Feb	13-Feb	13-Feb	17-Feb	100%	
61		Création de l'arbre (POO)	Kateryna,Diana	13-Feb	13-Feb	13-Feb	17-Feb	100%	
62		Déclaration des variables globales	Kateryna,Diana	14-Feb	14-Feb	14-Feb	17-Feb	100%	
63		Expression	Mehmet, Alyssa	14-Feb	14-Feb	14-Feb	17-Feb	100%	
64		Construction de l'arbre	Mehmet, Alyssa	15-Feb	15-Feb	15-Feb	17-Feb	100%	
65		gestion des opérateurs (+,-,*,>=,<=,!=,==)	Mehmet, Alyssa	15-Feb	15-Feb	15-Feb	17-Feb	100%	
66		Ecriture(1 et 2)	Tout le monde	15-Feb	15-Feb	15-Feb	17-Feb	100%	
67		Variables locales et paramètres	Tout le monde	15-Feb	15-Feb	15-Feb	17-Feb	100%	
68		Traitement du calcul des expressions	Tout le monde	16-Feb	16-Feb	16-Feb	17-Feb	100%	
69		Création de 2 piles (variable et paramètre)	Tout le monde	16-Feb	16-Feb	16-Feb	17-Feb	100%	
70		Conditionnelle	Kateryna	16-Feb	16-Feb	16-Feb	17-Feb	100%	
71		Itération	Diana	16-Feb	16-Feb	17-Feb	17-Feb	100%	
72		Récursivité	Mehmet,Alyssa	16-Feb	16-Feb	17-Feb	17-Feb	100%	
73									

Annexe 5 : Grammaire enrichie du langage

```
package generated.fr.l3.miashs;

import fr.ul.miashs.compil.arbre.*;
import java.util.ArrayList;
import fr.l3.miashs.tds.*;

/* code Java inséré dans le parser */
parser code {:
   public Tds getTds() {
       return action_obj.constructeurTds.getTds();
   }
:}
action code {:
   private ConstructeurTds constructeurTds = new ConstructeurTds();

   private Fonction construireFonction(String nom, Bloc bloc) {
       Fonction f = new Fonction(nom);
       if (bloc != null && bloc.getFils() != null) {
           for (Noeud n : bloc.getFils()) {
               f.ajouterUnFils(n);
           }
       }
       return f;
   }

   private String fonctionCourante = null;
   private int rangLocal = 0;

   private void commencerFonction(String nom, String type, ArrayList params) {
       fonctionCourante = nom;
       rangLocal = 0;

       constructeurTds.ajouterFonction(nom, type);

       int rangParam = 0;
       for (Object o : params) {
           String nomParam = (String) o;
           constructeurTds.ajouterParam(nom, nomParam, rangParam);
           rangParam++;
       }
       constructeurTds.setNbParamFonction(nom, params.size());
   }

   private void finirFonction() {
       if (fonctionCourante != null) {
           constructeurTds.setNbVarFonction(fonctionCourante, rangLocal);
       }
       fonctionCourante = null;
       rangLocal = 0;
   }

   private void ajouterLocale(String nom) {
       constructeurTds.ajouterLocal(fonctionCourante, nom, rangLocal);
       rangLocal++;
   }
:}

/* GRAMMAIRE */
/* 1. terminaux */
terminal ENTIER, VIDE;
```

```
terminal SI, SINON, TANTQUE, RETOURNER, ECRIRE, LIRE;

terminal PLUS, MOINS, MUL, DIV;
terminal AFF;
terminal SUP, INF, SUPE, INFE, EG, DIF;

terminal PO, PF, AO, AF, PV, VIRG;

terminal Integer NOMBRE;
terminal String IDF;

/* 2. non terminaux */
non terminal Prog programme;
non terminal Prog liste_definitions;
non terminal Fonction definition;

non terminal ArrayList liste_parametres_opt;
non terminal ArrayList liste_parametres;
non terminal String parametre;

non terminal Bloc bloc;
non terminal ArrayList declarations_locales_opt;
non terminal ArrayList declarations_locales;
non terminal String declaration_locale;

non terminal Bloc liste_instructions_opt;
non terminal Bloc liste_instructions;
non terminal Noeud instruction;

non terminal Affectation affectation;
non terminal Ecrire ecriture;
non terminal Retour retour;
non terminal Si alternative;
non terminal TantQue repetitive;
non terminal Appel appel;

non terminal ArrayList liste_arguments_opt;
non terminal ArrayList liste_arguments;

non terminal Noeud condition;
non terminal Noeud expression;
non terminal Noeud terme;
non terminal Noeud facteur;

/* priorités */
precedence left PLUS, MOINS;
precedence left MUL, DIV;

/* 3. axiome / start */
start with programme;

/* 4. REGLES */
programme ::= liste_definitions:ld
{: RESULT = ld; :}
;

liste_definitions ::= liste_definitions:ld definition:d
{:
   if (d != null) {
       ld.ajouterUnFils(d);
   }
   RESULT = ld;
```

```
:}
| definition:d
{:
   Prog p = new Prog();
   if (d != null) {
       p.ajouterUnFils(d);
   }
   RESULT = p;
:}
;

/* définitions globales + fonctions */
definition ::= ENTIER IDF:id PV
{:
   constructeurTds.ajouterGlobale(id);
   RESULT = null;
:}
| ENTIER IDF:id AFF expression:e PV
{:
   Integer val = null;
   if (e instanceof Const) {
       val = ((Const)e).getValeur();
   }
   constructeurTds.ajouterGlobale(id, val);
   RESULT = null;
:}
| ENTIER IDF:id PO liste_parametres_opt:lp PF
 {:
     commencerFonction(id, "int", lp);
 :}
 bloc:b
 {:
     RESULT = construireFonction(id, b);
     finirFonction();
 :}
| VIDE IDF:id PO liste_parametres_opt:lp PF
 {:
     commencerFonction(id, "void", lp);
 :}
 bloc:b
 {:
     RESULT = construireFonction(id, b);
     finirFonction();
 :}
;

/* paramètres */
liste_parametres_opt ::= liste_parametres:lp
{: RESULT = lp; :}
|
{:
   RESULT = new ArrayList();
:}
;

liste_parametres ::= liste_parametres:lp VIRG parametre:p
{:
   lp.add(p);
   RESULT = lp;
:}
| parametre:p
{:
```

```
   ArrayList l = new ArrayList();
   l.add(p);
   RESULT = l;
:}
;

parametre ::= ENTIER IDF:id
{: RESULT = id; :}
;

/* bloc */
bloc ::= AO declarations_locales_opt liste_instructions_opt:li AF
{: RESULT = li; :}
;

/* variables locales */
declarations_locales_opt ::= declarations_locales:dl
{: RESULT = dl; :}
|
{:
   RESULT = new ArrayList();
:}
;

declarations_locales ::= declarations_locales:dl declaration_locale:d
{:
   dl.add(d);
   RESULT = dl;
:}
| declaration_locale:d
{:
   ArrayList l = new ArrayList();
   l.add(d);
   RESULT = l;
:}
;

declaration_locale ::= ENTIER IDF:id PV
{:
   ajouterLocale(id);
   RESULT = id;
:}
;

/* instructions */
liste_instructions_opt ::= liste_instructions:li
{: RESULT = li; :}
|
{:
   RESULT = new Bloc();
:}
;

liste_instructions ::= liste_instructions:li instruction:i
{:
   li.ajouterUnFils(i);
   RESULT = li;
:}
| instruction:i
{:
   Bloc b = new Bloc();
   b.ajouterUnFils(i);
```

```
   RESULT = b;
:}
;

instruction ::= affectation:a PV
{: RESULT = a; :}
| ecriture:e PV
{: RESULT = e; :}
| appel:a PV
{: RESULT = a; :}
| retour:r PV
{: RESULT = r; :}
| alternative:s
{: RESULT = s; :}
| repetitive:t
{: RESULT = t; :}
;

/* affectation */
affectation ::= IDF:id AFF expression:e
{:
   Affectation a = new Affectation();
   a.setFilsGauche(new Idf(id));
   a.setFilsDroit(e);
   RESULT = a;
:}
;

/* écriture */
ecriture ::= ECRIRE PO expression:e PF
{:
   Ecrire ecr = new Ecrire();
   ecr.setLeFils(e);
   RESULT = ecr;
:}
;

/* retour */
retour ::= RETOURNER expression:e
{:
   Retour r = new Retour(fonctionCourante);
   r.setLeFils(e);
   RESULT = r;
:}
;

/* alternative */
alternative ::= SI PO condition:c PF bloc:alors
{:
   Si s = new Si();
   s.setCondition(c);
   s.setBlocAlors(alors);
   s.setBlocSinon(new Bloc());
   RESULT = s;
:}
| SI PO condition:c PF bloc:alors SINON bloc:sinon
{:
   Si s = new Si();
   s.setCondition(c);
   s.setBlocAlors(alors);
   s.setBlocSinon(sinon);
   RESULT = s;
```

```
:}
;

/* répétitive */
repetitive ::= TANTQUE PO condition:c PF bloc:b
{:
   TantQue t = new TantQue();
   t.setCondition(c);
   t.setBloc(b);
   RESULT = t;
:}
;

/* appel */
appel ::= IDF:id PO liste_arguments_opt:la PF
{:
   Appel a = new Appel(id);
   for (Object o : la) {
       a.ajouterUnFils((Noeud) o);
   }
   RESULT = a;
:}
;

liste_arguments_opt ::= liste_arguments:la
{: RESULT = la; :}
|
{:
   RESULT = new ArrayList();
:}
;

liste_arguments ::= liste_arguments:la VIRG expression:e
{:
   la.add(e);
   RESULT = la;
:}
| expression:e
{:
   ArrayList l = new ArrayList();
   l.add(e);
   RESULT = l;
:}
;

/* condition */
condition ::= expression:g SUP expression:d
{:
   Superieur n = new Superieur();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
| expression:g INF expression:d
{:
   Inferieur n = new Inferieur();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
| expression:g SUPE expression:d
{:
```

```
   SuperieurEgal n = new SuperieurEgal();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
| expression:g INFE expression:d
{:
   InferieurEgal n = new InferieurEgal();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
| expression:g EG expression:d
{:
   Egal n = new Egal();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
| expression:g DIF expression:d
{:
   Different n = new Different();
   n.setFilsGauche(g);
   n.setFilsDroit(d);
   RESULT = n;
:}
;

/* expressions */
expression ::= expression:g PLUS terme:d
{:
   Plus p = new Plus();
   p.setFilsGauche(g);
   p.setFilsDroit(d);
   RESULT = p;
:}
| expression:g MOINS terme:d
{:
   Moins m = new Moins();
   m.setFilsGauche(g);
   m.setFilsDroit(d);
   RESULT = m;
:}
| terme:t
{: RESULT = t; :}
;

terme ::= terme:g MUL facteur:d
{:
   Multiplication m = new Multiplication();
   m.setFilsGauche(g);
   m.setFilsDroit(d);
   RESULT = m;
:}
| terme:g DIV facteur:d
{:
   Division d1 = new Division();
   d1.setFilsGauche(g);
   d1.setFilsDroit(d);
   RESULT = d1;
:}
| facteur:f
```

```
{: RESULT = f; :}
;

facteur ::= NOMBRE:n
{:
   RESULT = new Const(n);
:}
| IDF:id
{:
   RESULT = new Idf(id);
:}
| appel:a
{: RESULT = a; :}
| LIRE PO PF
{:
   RESULT = new Lire();
:}
| PO expression:e PF
{: RESULT = e; :}
;
```

Annexe 6 : Classe ConstructeurTds.java

```java
package fr.l3.miashs.tds;

/**
* Classe utilitaire pour construire progressivement la table des symboles
* pendant l'analyse syntaxique
*/
public class ConstructeurTds {

   private final Tds tds;

   public ConstructeurTds() {
       this.tds = new Tds();
   }

   public Tds getTds() {
       return tds;
   }

   //declarations globales
   public void ajouterGlobale(String nom) {
       Item item = new Item(nom, "int", CategorieSymbole.GLOBAL);
       tds.ajouter(item);
   }

   public void ajouterGlobale(String nom, Integer val) {
       Item item = new Item(nom, "int", CategorieSymbole.GLOBAL);
       item.setVal(val);
       tds.ajouter(item);
   }

   //fonctions
   public void ajouterFonction(String nom, String type) {
       Item item = new Item(nom, type, CategorieSymbole.FONCTION);
       tds.ajouter(item);
   }

   public void setNbParamFonction(String nomFonction, int nbParam) {
       if (nbParam == 0) return;
       Item item = tds.rechercher(nomFonction);
       verifierFonction(item, nomFonction);
       item.setNbParam(nbParam);
   }

   public void setNbVarFonction(String nomFonction, int nbVar) {
       if (nbVar == 0) return;
       Item item = tds.rechercher(nomFonction);
       verifierFonction(item, nomFonction);
       item.setNbVar(nbVar);
   }

   //param

   public void ajouterParam(String nomFonction, String nomParam, int rang) {
       String nomComplet = nomParam + "@" + nomFonction;

       Item item = new Item(nomComplet, "int", CategorieSymbole.PARAM);
       item.setScope(nomFonction);
       item.setRang(rang);

       tds.ajouter(item);
```

```
    }

    //variables locales

    public void ajouterLocal(String nomFonction, String nomLocal, int rang) {
        String nomComplet = nomLocal + "@" + nomFonction;

        Item item = new Item(nomComplet, "int", CategorieSymbole.LOCAL);
        item.setScope(nomFonction);
        item.setRang(rang);

        tds.ajouter(item);
    }

    //verifications
    private void verifierFonction(Item item, String nomFonction) {
        if (item == null) {
            throw new IllegalArgumentException("Fonction non trouvée dans la TDS : " +
nomFonction);
        }
        if (item.getCategorie() != CategorieSymbole.FONCTION) {
            throw new IllegalArgumentException("Le symbole '" + nomFonction + "' n'est pas
une fonction");
        }
    }
}
```

Annexe 7 : Résultat de l'exemple 1 : Programme minimal

```
OK !
src/main/resources/samples/ex1_programme_minimal.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 8 : Résultat de l'exemple 2 : Variables globales

```
OK !
src/main/resources/samples/ex2_variables_globales.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=i, type=int, cat=GLOBAL, val=10 }
{ nom=j, type=int, cat=GLOBAL, val=20 }
{ nom=k, type=int, cat=GLOBAL }
{ nom=l, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        i : LONG(10)
        j : LONG(20)
        k : LONG(0)
        l : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 9 : Résultat de l'exemple 3 : Expression

```
OK !
src/main/resources/samples/ex3_expression.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─AFF
        └─IDF/x
        └─PLUS
            └─MUL
                └─IDF/a
                └─CONST/2
            └─DIV
                └─MOINS
                    └─IDF/b
                    └─CONST/5
                └─CONST/3

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=x, type=int, cat=GLOBAL, val=0 }
{ nom=a, type=int, cat=GLOBAL, val=100 }
{ nom=b, type=int, cat=GLOBAL, val=170 }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        x : LONG(0)
        a : LONG(100)
        b : LONG(170)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        LD(a, R0)
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        LD(b, R0)
```

```
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 10 : Résultat de l'exemple 4 : Ecriture

```
OK !
src/main/resources/samples/ex4_ecriture.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─AFF
        └─IDF/x
        └─PLUS
            └─MUL
                └─LIRE
                └─CONST/2
            └─DIV
                └─MOINS
                    └─LIRE
                    └─CONST/5
            └─CONST/3
    └─ECR
        └─IDF/x

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=x, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        x : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        RDINT()
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        RDINT()
        PUSH(R0)
        CMOVE(5, R0)
```

```
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
        LD(x, R0)
        PUSH(R0)
        POP(R0)
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 11 : Résultat de l'exemple 5 : Ecriture

```
OK !
src/main/resources/samples/ex5_ecriture.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─ECR
        └─PLUS
            └─MUL
                └─IDF/a
                └─CONST/2
        └─DIV
            └─MOINS
                └─IDF/b
                └─CONST/5
            └─CONST/3

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=a, type=int, cat=GLOBAL, val=100 }
{ nom=b, type=int, cat=GLOBAL, val=170 }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(100)
        b : LONG(170)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        LD(a, R0)
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        LD(b, R0)
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
```

```
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        WRINT()
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 12 : Résultat de l'exemple 6 : Variables locales et paramètres

```
OK !
src/main/resources/samples/ex6_variables_locales_et_parametres.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/f
    └─AFF
        └─IDF/res
        └─PLUS
            └─MUL
                └─IDF/a
                └─CONST/2
            └─DIV
                └─MOINS
                    └─IDF/b
                    └─CONST/5
                └─CONST/3
    └─RET/f
        └─IDF/res
└─FONCTION/main
    └─ECR
        └─APPEL/f
            └─IDF/a
            └─IDF/c

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=f, type=int, cat=FONCTION, nb_param=2, nb_var=1 }
{ nom=a, type=int, cat=GLOBAL, val=100 }
{ nom=c, type=int, cat=GLOBAL, val=170 }
{ nom=a@f, type=int, cat=PARAM, rang=0, scope=f }
{ nom=b@f, type=int, cat=PARAM, rang=1, scope=f }
{ nom=res@f, type=int, cat=LOCAL, rang=0, scope=f }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(100)
        c : LONG(170)
debut:
        CALL(main)
        HALT()
f:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(1)
```

```
        GETFRAME(-16, R0)
        PUSH(R0)
        CMOVE(2, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        MUL(R0, R1, R0)
        PUSH(R0)
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(5, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CMOVE(3, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        DIV(R0, R1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, 0)
        GETFRAME(0, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -20)
        BR(ret_f)
ret_f:
        DEALLOCATE(1)
        POP(BP)
        POP(LP)
        RTN()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        ALLOCATE(1)
        LD(a, R0)
        PUSH(R0)
        LD(c, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(2)
        POP(R0)
        WRINT()
```

```
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 13 : Résultat de l'exemple 7 : Conditionnelle

```
OK !
src/main/resources/samples/ex7_conditionnelle.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─SI/0
        └─SUP
            └─IDF/a
            └─IDF/b
        └─BLOC
            └─AFF
                └─IDF/x
                └─CONST/1000
        └─BLOC
            └─AFF
                └─IDF/x
                └─CONST/2000

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=a, type=int, cat=GLOBAL, val=1 }
{ nom=b, type=int, cat=GLOBAL, val=2 }
{ nom=x, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        a : LONG(1)
        b : LONG(2)
        x : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
si_0:
        LD(a, R0)
        PUSH(R0)
        LD(b, R0)
        PUSH(R0)
        POP(R1)
        POP(R2)
        CMPLT(R1, R2, R3)
```

```
        PUSH(R3)
        POP(R0)
        BF(R0, sinon_0)
        CMOVE(1000, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
        BR(fsi_0)
sinon_0:
        CMOVE(2000, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, x)
fsi_0:
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 14 : Résultat de l'exemple 8 : Itération

```
OK !
src/main/resources/samples/ex8_iteration.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/main
    └─AFF
        └─IDF/i
        └─CONST/0
    └─TQ/0
        └─INF
            └─IDF/i
            └─CONST/6
        └─BLOC
            └─ECR
                └─IDF/i
            └─AFF
                └─IDF/i
                └─PLUS
                    └─IDF/i
                    └─CONST/1

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=i, type=int, cat=GLOBAL }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
        i : LONG(0)
debut:
        CALL(main)
        HALT()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        CMOVE(0, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, i)
tq_0:
        LD(i, R0)
        PUSH(R0)
        CMOVE(6, R0)
        PUSH(R0)
```

```
        POP(R1)
        POP(R2)
        CMPLT(R2, R1, R3)
        PUSH(R3)
        POP(R0)
        BF(R0, ftq_0)
        LD(i, R0)
        PUSH(R0)
        POP(R0)
        WRINT()
        LD(i, R0)
        PUSH(R0)
        CMOVE(1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        ST(R0, i)
        BR(tq_0)
ftq_0:
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```

Annexe 15 : Résultat de l'exemple 9 : Récursivité

```
OK !
src/main/resources/samples/ex9_recursivite.txt analysé avec succès.

========== ARBRE ABSTRAIT ==========
PROG
└─FONCTION/f
  └─SI/0
    └─INFE
      └─IDF/a
      └─CONST/0
    └─BLOC
      └─RET/f
        └─CONST/0
    └─BLOC
  └─RET/f
    └─PLUS
      └─IDF/a
      └─APPEL/f
        └─MOINS
          └─IDF/a
          └─CONST/1
└─FONCTION/main
  └─ECR
    └─APPEL/f
      └─CONST/6

========== TABLE DES SYMBOLES ==========
TDS
{ nom=main, type=void, cat=FONCTION }
{ nom=f, type=int, cat=FONCTION, nb_param=1 }
{ nom=a@f, type=int, cat=PARAM, rang=0, scope=f }

========== CODE GENERE ==========
.include beta.uasm
.options tty
.include intio.uasm
        CMOVE(pile, SP)
        BR(debut)
debut:
        CALL(main)
        HALT()
f:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
si_0:
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(0, R0)
        PUSH(R0)
```

```
        POP(R1)
        POP(R2)
        CMPLT(R1, R2, R3)
        CMPEQ(R3, R31, R3)
        PUSH(R3)
        POP(R0)
        BF(R0, sinon_0)
        CMOVE(0, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -16)
        BR(ret_f)
        BR(fsi_0)
sinon_0:
fsi_0:
        GETFRAME(-12, R0)
        PUSH(R0)
        ALLOCATE(1)
        GETFRAME(-12, R0)
        PUSH(R0)
        CMOVE(1, R0)
        PUSH(R0)
        POP(R1)
        POP(R0)
        SUB(R0, R1, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(1)
        POP(R1)
        POP(R0)
        ADD(R0, R1, R0)
        PUSH(R0)
        POP(R0)
        PUTFRAME(R0, -16)
        BR(ret_f)
ret_f:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
main:
        PUSH(LP)
        PUSH(BP)
        MOVE(SP, BP)
        ALLOCATE(0)
        ALLOCATE(1)
        CMOVE(6, R0)
        PUSH(R0)
        CALL(f)
        DEALLOCATE(1)
        POP(R0)
        WRINT()
```

```
ret_main:
        DEALLOCATE(0)
        POP(BP)
        POP(LP)
        RTN()
pile:
        STORAGE(200)

Terminé !
```